

NIIKE

Algorithm Specifications and Supporting Documentation

June 30, 2026

Principal submitter: Yunlei Zhao

E-mail address: ylvzhao@fudan.edu.cn

Telephone: +86 13564812886

Affiliation: Fudan University

Postal address:

No. 2005, Songhu Road, Yangpu District,
Shanghai, China

Auxiliary submitters:

- Kaizhan Lin, Fudan University
- Weize Wang, Fudan University
- Chutong Wang, Fudan University
- Qifan Xue, Fudan University
- Zhi Hu, Central South University
- Zheng Tao, Central South University
- Yijing Zhang, Central South University
- Zhaohui Cheng, Shenzhen Olym Information Security Technology Co., Ltd

Inventors/developers (in alphabetical order): Kaizhan Lin, Weize Wang and Yunlei Zhao

Contents

1	Introduction	1
1.1	Background	1
1.2	Sketch of NIIKE	2
1.3	Protocol Features	3
2	Preliminary	5
2.1	Finite Fields	5
2.2	Elliptic Curves	6
2.3	Isogenies	9
2.4	Quaternion Algebra	11
3	Group action	14
3.1	Notations	14
3.2	Orientation	14
3.3	Class Group Action	15
3.4	Kernel Representations	15
3.5	Class Group Action Tailored to NIIKE	16
3.6	Tail Pruning	17
4	Specification of NIIKE	19
4.1	Setup	19
4.2	Key Generation	19
4.3	Key Agreement	20
4.4	Theoretical Key Size	21
5	Parameters	24
5.1	Parameter Requirements	24
5.2	Characteristic Finding	25
5.3	Orientation Finding	26
5.4	Kernel Representation Construction	28

5.5	Parameter Sets	32
6	Test Vectors	36
7	Performance analysis	37
7.1	Key Sizes	37
7.2	Performance Evaluation	37
8	Implementation details	39
8.1	Finite Field Arithmetic	39
8.2	Elliptic Curve Arithmetic	39
8.3	Isogenies in Dimension One	42
8.4	Parallel Optimizations	46
9	Security	51
9.1	Theoretical Security	51
9.2	Practical Security	53
9.3	Other Security Properties	55
10	Failure Analysis	56

CHAPTER 1

Introduction

1.1 Background

As a branch of post-quantum cryptography, isogeny-based cryptography leverages the mathematical structure of elliptic curves and the isogenies between elliptic curves that preserve group structure. Unlike traditional elliptic curve cryptography, which relies on the difficulty of solving discrete logarithms in an elliptic curve group, isogeny-based cryptography takes advantage of the hardness of finding isogenies between elliptic curves. On the other hand, similar to traditional elliptic curve cryptography, isogeny-based protocols offer small public keys, making it appealing for resource-constrained environments, where bandwidth and storage are limited.

Non-Interactive Key Exchange (NIKE), as a fundamental cryptographic primitive, exhibits significant advantages over traditional interactive key-establishment mechanisms based on key encapsulation mechanisms (KEMs) in terms of both performance and applicability within modern network protocols and secure communication architectures. First, with respect to communication overhead and latency optimization, NIKE enables each party to derive a shared key independently and unilaterally using only its own secret key and the counterparty's public key, thereby completely eliminating the additional communication round-trips required by conventional handshake protocols. This property is particularly critical in OPTLS, KEMTLS, and HTTPS scenarios that require early data transmission, as it effectively avoids handshake latency caused by waiting for full server authentication. Second, in asynchronous and offline communication environments, NIKE possesses an indispensable advantage that interactive protocols cannot readily replace. When the communicating parties cannot be assumed to be online simultaneously, two-stage KEM-based mechanisms become fundamentally difficult to deploy, whereas NIKE allows an initiator to establish a session securely without the real-time participation of the peer. For example, in X3DH, a core component of the signal secure-messaging protocol, it is precisely the use of static and semi-static non-interactive key exchange that makes it possible to achieve multiple advanced security properties simultaneously, includ-

ing forward secrecy, mutual authentication, and offline deniability, without requiring direct interaction between both parties. As a consequence, non-interactive key exchange is of substantial engineering value not only because it reduces the network interaction cost of secure protocols, but also because it serves as an indispensable cornerstone for modern asynchronous secure communication systems. **However, nearly all existing PQC-based key exchange mechanisms, inherently rely on interactive protocols, with non-interactive variants remaining either theoretically unproven or practically infeasible.** To our knowledge, only lattice-based protocol Swoosh [GdKQ⁺24] and some isogeny-based protocols such as CSIDH [CLM⁺18] are non-interactive. However, Swoosh has huge public-key size (220 KBs targeting ≈ 120 -bit quantum security label), while CSIDH and its variants [BBC⁺21, CCSC⁺24, CHMR25, BHS25] suffer from inefficiency because of large characteristic of the base finite field (4096 bit) targeting 128-bit classical security label.

This specification presents a detailed description of the **Non-Interactive Isogeny-based Key Exchange**, referred to as **NIIKE**, including an overview of the mathematical foundations, a complete description of all algorithms and data type conversions used in the implementation, and a discussion of the protocol’s security. The underlying ideal class group action was first introduced by Houben in 2025 [Hou25a, Hou25b], and recently Wang et al. [WLLZ26] optimized its performance in theory and implementation. NIIKE is a **non-interactive key exchange**, which is an extremely rare attribute in post-quantum cryptography. Besides, NIIKE is **dummy-free, constant-time, and deterministic**. Moreover, the public key is **much more compact** compared to Swoosh, and has smaller characteristic compared to CSIDH targeting classical 128 and 256 security levels. Finally, according to the research of [WLLZ26], the novel isogeny group action underlying NIIKE is **well-suited for parallel implementations** and has significant potential for further optimization.

1.2 Sketch of NIIKE

NIIKE is based on a novel ideal class group action, which has the dummy-free, non-interactive, and deterministic properties. Given an elliptic curve with a fixed orientation $\mathcal{E}$, the algorithm outputs another elliptic curve by keeping track of the orientation, according to a vector which corresponds to a specific ideal class group action, acting as the secret key in NIIKE. In the implementation, we use a set of curves and rational points defined on the curves to represent a curve with a specific orientation.

Figure 1.1 illustrates a sketch of NIIKE. In key generation, each party selects a vector as its secret key and then computes the corresponding class group action to derive the public key. After receiving the other party’s public key, each party applies the class group action with respect to its secret key during key agreement and ultimately obtains an isomorphic curve $E_{AB} \cong E_{BA}$. They take the j -invariant of the curves $j(E_{AB}) = j(E_{BA})$ as the shared secret.

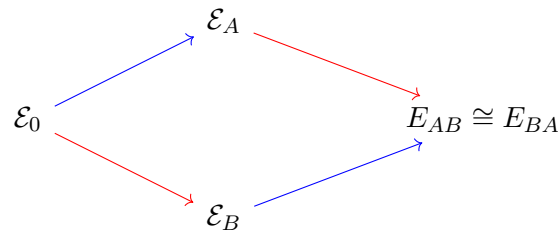


Figure 1.1: A sketch of NIIKE

1.3 Protocol Features

1.3.1 Innovativeness

The innovativeness of NIIKE lies in its novel ideal class group action. Distinct from CSIDH, NIIKE uses a base field of small characteristic in implementations, thereby reducing the public key size and improving overall performance. Moreover, the ideal class group action is dummy-free, deterministic, and constant-time, and thus NIIKE has the strong security against side-channel attacks in theory. Furthermore, based on our work [WLLZ26] (accepted by TCHES 2026), we improve the performance of NIIKE and introduce new parallel strategies, making it practical in applications.

1.3.2 Simplicity

It is easy to understand NIIKE at a high level, as it is quite similar to traditional Diffie-Hellman. The key generation and key agreement algorithms are very similar, and they simply apply the group action algorithm several times. The subroutines of the group action algorithm only include basic elliptic curve arithmetic and one-dimensional isogeny computations, which are the usual operations in isogeny-based protocols. Since our implementation does not involve two-dimensional isogeny computations and complicated quaternion operations, our implementation is not challenging for beginners to comprehend.

1.3.3 Flexibility, compatibility and Extensibility

For low security levels, NIIKE demonstrates flexibility through its support for parameter agility. The scheme allows for seamless adjustments of security levels, including varying the base field size or the discriminant of the orientation, without necessitating a change in the core algorithmic structure. This enables users to balance the trade-off between computational overhead and long-term cryptographic security according to their specific needs. For high security levels, NIIKE suffers from the hardness of finding a small p such that $p^2 - 1$ is smooth and the kernel representation generation of the orientation (See Sections 5.2 to 5.4 for more details).

NIIKE is designed with a considerable degree of protocol compatibility and extensibility, ensuring it can be integrated into existing security architectures with minimal friction. Note that NIIKE shares many similarities with traditional Diffie-Hellman algorithms in terms of cryptographic architecture. Therefore, some research on cryptographic design based on traditional Diffie-Hellman algorithms can also be applied to the architecture of NIIKE. For example, NIIKE can be seamlessly adapted from two-party key exchange to more complex scenarios, including Multi-Party Key Agreement (MPKA) and Group Key Management, by leveraging existing research frameworks designed for traditional DH-based group communications. The handshake logic and data structures of NIIKE are engineered to be consistent with industry-standard protocols such as TLS 1.3, IKEv2, and SSH. This allows for a 'drop-in' replacement or enhancement of classical components within established communication frameworks.

1.3.4 Performance advantages on variant platforms

The main application bottleneck of NIIKE lies in efficiency. To close the gap between other post-quantum protocols and NIIKE, we will apply the fork-join parallel execution model to optimize the class group action performance in Section 8.4, and achieve a near perfect parallelism in key generation, and a better performance in key agreement. This inherent parallelism allows NIIKE to leverage various hardware acceleration techniques effectively. On modern general-purpose CPUs, these subroutines can be mapped to SIMD (Single Instruction, Multiple Data) instructions, such as AVX-512, enabling the simultaneous processing of multiple group action components within a single clock cycle. This significantly reduces the overall latency of the key exchange compared to purely sequential implementations.

Therefore, we believe that NIIKE would be a versatile candidate for diverse deployment scenarios with further research, ranging from high-traffic network gateways to low-power IoT sensors, where both timing efficiency and resource utilization are critical.

CHAPTER 2

Preliminary

This section provides the mathematical foundation of NIIKE, including finite fields, elliptic curves, isogenies and quaternions.

2.1 Finite Fields

2.1.1 Finite field $\mathbb{F}_p$

Elements of the prime field $\mathbb{F}_p$ are uniquely represented by the set of integers $\{0, \dots, p-1\}$. We define the following algebraic operations:

- **Basic operations:** Addition, multiplication, and their respective inverses are performed modulo p , with results uniquely mapped to the interval $[0, p-1]$.
- **Quadratic residuosity:** An element $a \in \mathbb{F}_p$ is a square if there exists $b \in \mathbb{F}_p$ such that $b^2 = a$. This is determined by the Legendre symbol $a^{(p-1)/2}$, which yields 1 if a is a quadratic residue and -1 otherwise.
- **Square root:** For primes $p \equiv 3 \pmod{4}$, the canonical square root of a quadratic residue $a \in \mathbb{F}_p$ is computed directly as:

$$\sqrt{a} = a^{\frac{p+1}{4}} \pmod{p}. \quad (2.1)$$

- **Ordering:** We define an ordering on $\mathbb{F}_p$ by comparing the lifted integer representatives in $[0, p-1]$.

2.1.2 Finite field $\mathbb{F}_{p^2}$

In NIIKE, the characteristic p has the property that $p \equiv 3 \pmod{4}$. Therefore, the field extension $\mathbb{F}_{p^2}$ is defined as $\mathbb{F}_p(i)$ with $i^2 + 1 = 0$. Elements of the field $\mathbb{F}_{p^2}$ are uniquely represented by $a = a_0 + a_1 \cdot i$ with $a_0, a_1 \in \mathbb{F}_p$.

- **Addition and additive inverse:** For $a, b \in \mathbb{F}_{p^2}$, their sum is $c = (a_0 + b_0) + (a_1 + b_1)i$, and the additive inverse of a is $-a = (-a_0) + (-a_1)i$.

- **Multiplication:** The product ab is given by $c = c_0 + c_1i$, where:

$$c_0 = a_0b_0 - a_1b_1 \quad \text{and} \quad c_1 = a_0b_1 + a_1b_0.$$

- **Multiplicative inverse:** For any non-zero $a \in \mathbb{F}_{p^2}$, the multiplicative inverse is $a^{-1} = (a_0N^{-1}) + (-a_1N^{-1})i$, where $N = a_0^2 + a_1^2 \in \mathbb{F}_p$ represents the norm of a .
- **Quadratic residuosity:** An element $a \in \mathbb{F}_{p^2}$ is a square if and only if its norm $n(a) = a^{p+1} = a_0^2 + a_1^2$ is a quadratic residue in $\mathbb{F}_p$.
- **Square root:** If a is a quadratic residue in $\mathbb{F}_p$, we use the square root defined in Equation (2.1). Otherwise, $-a$ must be a square in $\mathbb{F}_p$, and we define $\sqrt{a} = \sqrt{-a} \cdot i$. If $a \in \mathbb{F}_{p^2} \setminus \mathbb{F}_p$ is a square, its canonical square root is computed as

$$\sqrt{a} = (-i)^{\frac{1-\chi}{2}} S(a + \delta), \quad (2.2)$$

where $\delta = \sqrt{a_0^2 + a_1^2}$, $S = (2(a_0 + \delta))^{\frac{p-3}{4}}$, and $\chi = (2(a_0 + \delta))^{\frac{p-1}{2}}$.

- **Lexicographic ordering:** Elements are ordered by comparing their components. We say $a < b$ if and only if $a_0 < b_0$, or if $a_0 = b_0$ and $a_1 < b_1$.

2.2 Elliptic Curves

This section reviews the essential properties of supersingular elliptic curves in Montgomery form, which serve as the foundation for the NIIKE implementation. For a more comprehensive discussion of Montgomery curves and their specific computational advantages, we refer the reader to [CS18].

2.2.1 Montgomery curves

The Montgomery curve $E_{A,B}$ over $\mathbb{F}_q$ is an elliptic curve defined by the equation

$$By^2 = x^3 + Ax^2 + x. \quad (2.3)$$

where $A, B \in \mathbb{F}_q$ satisfying that $B(A^2 - 4) \neq 0$. The curve consists of the set of points (x, y) satisfying the equation in any algebraic extension of $\mathbb{F}_q$, together with the point at infinity 0_E . We use the notation $E_{A,B}/\mathbb{F}_q$ to specify the base field of definition, and let $E_{A,B}(\mathbb{F}_q)$ denote the group of $\mathbb{F}_q$ -rational points. In cases where $B = 1$, the curve is simply denoted by E_A , where A is referred to as the *Montgomery coefficient*. In this specification, we may use E to represent a generic Montgomery curve.

Two elliptic curves are considered isomorphic over $\mathbb{F}_q$ if there exists a transformation $(x, y) \mapsto (Cx + R, Dy)$, where $C, D \in \mathbb{F}_q^*$ and $R \in \mathbb{F}_q$, that maps one curve equation to the other. When two curves, $E_{A,B}$ and $E_{A',B'}$, fail to be isomorphic over the base field $\mathbb{F}_q$ but become isomorphic over a finite extension field, they are referred to as twists of

one another. A common example is the quadratic twist. Specifically, the curves $E_{A,B}$ and $E_{A,B'}$ are always quadratic twists, and they are isomorphic over $\mathbb{F}_q$ if and only if the ratio B/B' is a quadratic residue (a square) in $\mathbb{F}_q$. If B/B' is a non-square, the isomorphism requires an extension field to define the scaling factor $C = \sqrt{B/B'}$. For a supersingular elliptic curve E , we denote its twist by E^t for brevity, omitting explicit reference to the base field when it is understood from the context.

Let N represent the number of $\mathbb{F}_q$ -rational points on the curve (including the point at infinity 0_E). An elliptic curve $E_{A,B}$ is classified as supersingular if $N \equiv 1 \pmod{\text{char}(\mathbb{F}_q)}$. In this specification, we focus exclusively on supersingular curves defined over the quadratic extension $\mathbb{F}_{p^2}$, where $p \equiv 3 \pmod{4}$. Under these conditions, the point counts behave predictably based on the curve's isomorphism class. Any supersingular curve $E_A/\mathbb{F}_{p^2}$ (with $B = 1$) possesses exactly $(p+1)^2$ points, and the quadratic twists $E_{A'}/\mathbb{F}_{p^2}$, constructed using an arbitrary quadratic non-residue $\gamma \in \mathbb{F}_{p^2}$, are all isomorphic to one another and each contain exactly $(p-1)^2$ points.

This j -invariant characterizes the isomorphism classes of elliptic curves over the algebraic closure. For a Montgomery curve $E_{A,B}$, its j -invariant is defined as

$$j(E_{A,B}) = \frac{256(A^2 - 3)^3}{A^2 - 4}. \quad (2.4)$$

Specifically, two curves share the same j -invariant if and only if they are isomorphic or are twists of one another. For two isomorphic Montgomery curves $E_{A,B}$ and $E_{A',B'}$, the coordinate transformation $(x, y) \mapsto (Cx + R, Dy)$ that maps the former to the latter is defined by:

$$\begin{aligned} C &= \lambda^2 \frac{B'}{B}, \\ R &= \lambda^2 \frac{AB'}{3B} - \frac{A'}{3}, \\ D &= \lambda^3 \frac{B'}{B}, \\ \lambda &= \sqrt{\frac{B(2A'^3 - 9A')}{B'(2A^3 - 9A)}}. \end{aligned} \quad (2.5)$$

2.2.2 Curve Arithmetic

The set of points of a Montgomery curve, equipped with the addition operation, forms an abelian group. Under this addition law, the inverse of each point $P = (x, y)$ is defined as $-P = (x, -y)$, and the point at infinity 0_E is the identity element.

In the following, for a point $P \neq 0_E$, we refer to its x -coordinate as x_P , and to its y -coordinate as y_P , i.e., $P = (x_P, y_P)$. For efficient implementations, we typically use projective coordinates $(X : Y : Z)$ with $x = X/Z$ and $y = Y/Z$ in order to avoid inversions in the point addition and isogeny computations. Furthermore, in most cases one can use

the x -only arithmetic to represent points, i.e., $P = (X_P : Z_P)$, which means that points are only defined up to the sign of their y -coordinates.

2.2.2.1 Point addition

For any two points $P = (x_P, y_P)$ and $Q = (x_Q, y_Q)$ on $E_{A,B}$ such that $P \neq \pm Q$, their sum $R = P + Q = (x_R, y_R)$ is given by

$$x_R = B\lambda^2 - A - x_P - x_Q, \quad (2.6)$$

and

$$y_R = \lambda(x_P - x_R) - y_P, \quad (2.7)$$

where the slope λ is defined as $\lambda = (y_Q - y_P)/(x_Q - x_P)$. The point at infinity, denoted by 0_E , serves as the identity element of the group, satisfying $P + 0_E = 0_E + P = P$ for all $P \in E_{A,B}$.

2.2.2.2 Point doubling

Let $E_{A,B}/\mathbb{F}_q$ be an elliptic curve in Montgomery form. For any point $P = (x_P, y_P) \in E_{A,B} \setminus E[2]$, the coordinates of the doubled point $[2]P = (x_{[2]P}, y_{[2]P})$ are computed as follows:

$$x_{[2]P} = \frac{(x_P^2 - 1)^2}{4x_P(x_P^2 + Ax_P + 1)}, \quad (2.8)$$

and

$$y_{[2]P} = y_P \frac{(x_P^2 - 1)(x_P^4 + 2Ax_P^3 + 6x_P^2 + 2Ax_P + 1)}{8x_P^2(x_P^2 + Ax_P + 1)^2}. \quad (2.9)$$

In the case where $P = -P$, i.e., P is a point of order 2 or the point at infinity, the doubling operation results in the point at infinity, $[2]P = 0_E$.

2.2.2.3 Scalar multiplication

The abelian group law on an elliptic curve E induces a scalar multiplication map, denoted by $[k] : E \rightarrow E$ for any $k \in \mathbb{Z}$. For a point $P \in E$, the operation $[k]P$ is defined as the sum of k copies of P . When $k < 0$, we define $[k]P = [-k](-P)$. While for $k = 0$, the operation yields the point at infinity, i.e., $[0]P = 0_E$. To optimize performance, scalar multiplication is typically implemented as a sequence of point doublings and additions. By employing the Montgomery ladder (detailed in Section 8.2.1), the computational complexity is reduced to $O(\log k)$, making the cost proportional to the bit-length of the scalar. The order of a point P is the smallest positive integer m such that $[m]P = 0_E$.

2.2.2.4 Torsion subgroups

Let $E/\mathbb{F}_{p^2}$ be a supersingular curve and m an integer. The m -torsion subgroup, denoted by $E[m]$, consists of all points $P \in E(\overline{\mathbb{F}_{p^2}})$ such that $[m]P = 0_E$. Provided that m^2 divides the order of the group $\#E(\mathbb{F}_{p^2})$, the subgroup is isomorphic to the product of two cyclic groups:

$$E[m] \simeq \mathbb{Z}/m\mathbb{Z} \times \mathbb{Z}/m\mathbb{Z}. \quad (2.10)$$

Consequently, $E[m]$ has a cardinality of m^2 . Any such m -torsion subgroup can be generated by a pair of (non-unique) points $R, S \in E[m]$, such that $\langle R, S \rangle = E[m]$. We refer to this pair (R, S) as a basis for $E[m]$.

2.3 Isogenies

An isogeny $\varphi : E_1 \rightarrow E_2$ is a non-constant algebraic morphism that maps the identity element of E_1 to that of E_2 . Such a morphism is uniquely determined, up to isomorphism, by its kernel $\ker(\varphi) = \{P \in E_1 \mid \varphi(P) = 0_{E_2}\}$. In the context of NIKE, we focus on cyclic isogenies, whose kernels are cyclic groups and can thus be efficiently represented by a single generator. Furthermore, every isogeny of degree n admits a unique dual isogeny $\hat{\varphi} : E_2 \rightarrow E_1$, satisfying the property that the composition $\hat{\varphi} \circ \varphi$ is equivalent to the scalar multiplication map $[n]$ on E_1 .

An endomorphism is a special case of an isogeny where the domain and codomain curves are identical. The set of all endomorphisms forms the endomorphism ring $\text{End}(E)$, which provides the algebraic structure necessary for ideal class group actions in Chapter 3.

For the computation of an isogeny φ defined over $\mathbb{F}_q$, we can represent it using a pair of rational functions $f(x), g(x) \in \mathbb{F}_q(x)$ such that $\varphi(x, y) = (f(x), y \cdot g(x))$. These functions are expressed as ratios of coprime polynomials, e.g., $f(x) = f_1(x)/f_2(x)$, where the degree of the isogeny is given by $\deg(\varphi) = \max\{\deg(f_1), \deg(f_2)\}$. An isogeny is called separable if its degree is equal to the cardinality of its kernel. In the following, we use the term n -isogeny to denote a separable isogeny of degree n .

Given a point P of order ℓ coprime to the characteristic p , one can construct and evaluate an isogeny φ with kernel $\langle P \rangle$ by Vélú's formula [Vé71], which has the computational complexity $\tilde{O}(\ell)$. For efficiency, the degree N is set to be a smooth integer, i.e., $N = \prod_{i=1}^n \ell_i$ with ℓ_i small, so that the isogeny computations of φ can be decomposed into ℓ_i -isogeny computations. In particular, given a supersingular elliptic curve $E_1/\mathbb{F}_q : By^2 = x^3 + Ax^2 + x$ with $\text{char}(\mathbb{F}_q) = p > 2$, an isogeny of odd degree $\ell = 2d + 1$ with kernel $\langle P \rangle$ is defined by

$$\begin{aligned} \varphi : E_1 &\rightarrow E_2, \\ (x, y) &\mapsto (f(x), y \cdot f'(x)), \end{aligned} \quad (2.11)$$

where

$$f(x) = x \cdot \prod_{i=1}^d \left(\frac{x \cdot x_{[i]P} - 1}{x - x_{[i]P}} \right)^2, \quad (2.12)$$

and $f'(x)$ is its derivative [CH17]. The codomain of φ is defined by

$$E_2/\mathbb{F}_q : B'y^2 = x^3 + A'x^2 + x, \quad (2.13)$$

where $A' = \left(6 \sum_{i=1}^d \left(\frac{1}{x_{[i]P}} - x_{[i]P}\right) + A\right) \cdot \prod_{i=1}^d x_{[i]P}$ and $B' = B \cdot \prod_{i=1}^d x_{[i]P}$. For more isogeny computation details, we refer to Section 8.3.

For relatively large ℓ , the isogeny computation can be optimized by using $\sqrt{\ell}$'s formula [BDFLS20]. Let $h_S(x) = \prod_{i \in S} (x - x_{[i]P})$. Then Equation (2.12) can be written as

$$f(x) = x^\ell \cdot \prod_{i \in S} \left(\frac{h_S(1/x)}{h_S(x)} \right)^2, \quad (2.14)$$

where $S = \{1, 3, \dots, \ell - 2\}$. The coefficient A' can be computed by

$$A' = 2 \cdot \frac{1+d}{1-d}, \text{ where } d = \left(\frac{A+2}{A-2} \right)^\ell \left(\frac{h_S(1)}{h_S(-1)} \right)^8. \quad (2.15)$$

It is easy to see that the construction and evaluation of h_S determines the computational overhead of the isogeny computations. Define the set $S = \{1, 3, \dots, \ell - 2\}$ and partition it as $S = (I \pm J) \cup K$, where $I = \{2b + 4bi \mid 0 \leq i < b'\}$, $J = \{2j + 1 \mid 0 \leq j < b\}$ and $K = S \setminus (I \pm J)$. Let $b = \lfloor \sqrt{\ell - 1}/2 \rfloor$ and $b' = \lfloor (\ell - 1)/4b \rfloor$, then $I + J$ and $I - J$ are disjoint. To evaluate the polynomial $h_S(x)$, we first compute the x -coordinate sets $xI = \{x([i]P) \mid i \in I\}$, $xJ = \{x([j]P) \mid j \in J\}$ and $xK = \{x([k]P) \mid k \in K\}$. By applying the following formula:

$$(x - x_{P+Q})(x - x_{P-Q}) = x^2 + \frac{F_1(x_P, x_Q)}{F_0(x_P, x_Q)}x + \frac{F_2(x_P, x_Q)}{F_0(x_P, x_Q)}, \quad (2.16)$$

with

$$\begin{aligned} F_0(X_1, X_2) &= (X_1 - X_2)^2, \\ F_1(X_1, X_2) &= -2((X_1 X_2 + 1)(X_1 + X_2) + 2A X_1 X_2), \\ F_2(X_1, X_2) &= (X_1 X_2 - 1)^2, \end{aligned} \quad (2.17)$$

the polynomial $h_{I \pm J}(x)$ is expressed as a ration of two resultants of smaller polynomials:

$$h_{I \pm J}(x) = \frac{\text{Res}_z(h_I(z), E_J(x, z))}{\text{Res}_z(h_I(z), D_J(z))}, \quad (2.18)$$

where

$$E_J(x, z) = \prod_{j \in J} (F_0(z, x_{[j]P})x^2 + F_1(z, x_{[j]P})x + F_2(z, x_{[j]P})), \quad (2.19)$$

and

$$D_J(z) = \prod_{j \in J} F_0(z, x_{[j]P}). \quad (2.20)$$

The sizes of I and J satisfy $b, b' \approx \sqrt{\ell}$. Therefore, the computational complexity of $\sqrt{\ell}\text{lu}$'s formula is $\tilde{O}(\sqrt{\ell})$. Note that this analysis hides polylogarithmic factors, which can outweigh the improvement in the exponent when the degree ℓ is small. This explains why $\sqrt{\ell}\text{lu}$'s formula performs slightly worse when ℓ is very small. With the optimizations of [LWZZ25], $\sqrt{\ell}\text{lu}$'s formula performs better when $\ell \geq 73$.

2.4 Quaternion Algebra

A quaternion algebra over $\mathbb{Q}$ ramified only at p and ∞ has the form $B_{p,\infty} = \mathbb{Q} + \mathbb{Q}i + \mathbb{Q}j + \mathbb{Q}k$, where $i^2 = -q$, $j^2 = -p$, and $k = ij = -ji$ for some integer q . For any $\alpha = a_1 + a_2i + a_3j + a_4k \in B_{p,\infty}$, the canonical involution is the map sending α to $\bar{\alpha} = a_1 - a_2i - a_3j - a_4k$. The reduced trace and the reduced norm of α are defined, respectively, by

$$\begin{aligned} \text{tr}(\alpha) &= \alpha + \bar{\alpha} = 2a_1, \\ \text{Nrd}(\alpha) &= \alpha\bar{\alpha} = a_1^2 + a_2^2 + pa_3^2 + pa_4^2. \end{aligned}$$

An order in $B_{p,\infty}$ is a full-rank lattice and it is also a subring. We call an order maximal when it is not contained in any other larger order. The endomorphism rings of supersingular elliptic curves over $\overline{\mathbb{F}}_p$ are isomorphic to maximal orders in $B_{p,\infty}$.

Let $\mathcal{O}$ be a maximal order. A full-rank lattice $I \subset \mathcal{O}$ is a left $\mathcal{O}$ -ideal if $\mathcal{O}I \subset I$, and it is a right $\mathcal{O}$ -ideal if $I\mathcal{O} \subset I$. For any left ideal I of a maximal order $\mathcal{O}$ in $B_{p,\infty}$, define the left order and right order of I as

$$\begin{aligned} \mathcal{O}_L(I) &= \{x \in B_{p,\infty} \mid xI \subseteq I\}, \\ \mathcal{O}_R(I) &= \{x \in B_{p,\infty} \mid Ix \subseteq I\}. \end{aligned}$$

Note that $\mathcal{O}_L(I)$ and $\mathcal{O}_R(I)$ are also maximal orders. We say that I connects $\mathcal{O}_L(I)$ and $\mathcal{O}_R(I)$, and the corresponding Eichler order of I is defined as $\mathfrak{O} = \mathcal{O}_L(I) \cap \mathcal{O}_R(I)$. The reduced norm of I is given by $\text{Nrd}(I) = \gcd(\{\text{Nrd}(\alpha) \mid \alpha \in I\})$. The conjugate of I , denoted by $\bar{I}$, is the set of conjugates of elements of I satisfying $I\bar{I} = \text{Nrd}(I)\mathcal{O}_L(I)$ and $\bar{I}I = \text{Nrd}(I)\mathcal{O}_R(I)$. Two left ideals I and J in $\mathcal{O}$ are equivalent if there exists $\alpha \in B_{p,\infty}^\times$ such that $J = I\alpha$, and we denote the set of such classes by $\text{Cl}(\mathcal{O})$.

2.4.1 Deuring correspondence

As shown in Table 2.1, the Deuring correspondence establishes a categorical equivalence between left $\mathcal{O}$ -ideals I of reduced norm n and isogenies $\varphi : E \rightarrow E'$ of degree n , facilitated by the bijective mappings $I \mapsto \varphi_I$ and $\varphi \mapsto I_\varphi$. Under this framework, the endomorphism

ring of the codomain is isomorphic to the right order within the quaternion algebra $B_{p,\infty}$. Specifically, an isogeny φ acts as an endomorphism of E if and only if its corresponding ideal $I = \mathcal{O}\varphi$ is principal. Furthermore, for any two isogenies $\varphi_1 : E \rightarrow E_1$ and $\varphi_2 : E \rightarrow E_2$ respectively associated with left $\mathcal{O}$ -ideals I_1 and I_2 , the codomain curves E_1 and E_2 are isomorphic if and only if I_1 and I_2 are equivalent within the same left ideal class.

Table 2.1: Summary of the Deuring correspondence

Curves & Isogenies	Quaternion Algebra
Isogeny $\varphi : E \rightarrow E'$	Left $\mathcal{O}$ -ideal I
Degree $\deg(\varphi) = n$	Reduced norm $n(I) = n$
Endomorphism ring $\text{End}(E')$	Right order $\mathcal{O}_R(I) = \{\alpha \in B_{p,\infty} \mid I\alpha \subseteq I\}$
Endomorphism $\varphi \in \text{End}(E)$	Principal left ideal $I = \mathcal{O}\varphi$
Isomorphic curves $E_1 \cong E_2$	Ideal equivalence $I_1 = I_2\alpha, \alpha \in B_{p,\infty}^\times$
Isogeny classes of E	Left ideal classes of $\mathcal{O}$

Note that for supersingular elliptic curves, $\text{End}(E)$ is isomorphic to a maximal order in a quaternion algebra. This structure allows us to link ideal classes of the endomorphism ring to specific isogenies, forming the mathematical basis for the ideal class group action used in protocols like NIIKE to facilitate secure cryptographic operations.

Remark 2.4.1. Let $p \equiv 3 \pmod{4}$ and $k \in \mathbb{N}$. Then the supersingular elliptic curve $E_0/\mathbb{F}_{p^k}$, defined by the equation $y^2 = x^3 + x$, has j -invariant 1728. The endomorphism ring of E_0 is isomorphic to the maximal order $\mathcal{O}_0 = \mathbb{Z} + \mathbb{Z}i + \mathbb{Z}\frac{i+j}{2} + \mathbb{Z}\frac{1+k}{2}$ in the quaternion algebra $B_{p,\infty}$, where the generators satisfy $i^2 = -1, j^2 = -p$, and $ij = -ji = k$. Specifically, the Frobenius endomorphism $\pi : (x, y) \mapsto (x^p, y^p)$ corresponds to the element j , while the distortion map $\omega : (x, y) \mapsto (-x, iy)$ is identified with i .

2.4.2 Cornacchia's Algorithm

Cornacchia's algorithm [Cor08] provides an efficient method for determining integer solutions to equations of the form $x^2 + y^2 = m$, where $m \in \mathbb{N}$, assuming the factorization of m is known. This algorithm is integrated into the setup phase of NIIKE.

Algorithm 1 CORNACCHIA(m)

Input: An integer $m \in \mathbb{Z}$ **Output:** x, y such that $x^2 + y^2 = m$, a boolean **found** which indicates whether a solution is found

- 1: $x \leftarrow 0, y \leftarrow 0, \text{found} \leftarrow 0$
 - 2: Find the 100-smooth part m' of m using trial division
 - 3: **If** m/m' is prime **then**
 - 4: Find $x, y \in \mathbb{N}$ such that $x^2 + y^2 = m$, and set $\text{found} \leftarrow \text{true}$ if a solution was found
 - 5: **end If**
 - 6: **Return** x, y, found
-

CHAPTER 3

Group action

This chapter illustrates the specific ideal class group actions in NIKE, and proposes corresponding algorithms.

3.1 Notations

Define $[m_1, m_2]$ as the set of integers s such that $m_1 \leq s \leq m_2$, and define $[m_1, m_2]^n$ as the set of vectors $(s_1, s_2, \dots, s_n)$ such that $m_1 \leq s_i \leq m_2$. For simplicity, we write $(s_i)_{i=1}^n$ for a vector $(s_1, s_2, \dots, s_n)$. Vector addition is defined component-wise by $(a_i)_{i=1}^n + (b_i)_{i=1}^n := (a_i + b_i)_{i=1}^n$. We say that $(a_i)_{i=1}^n$ is a multiple of $(b_i)_{i=1}^n$ if there exists some $m \in \mathbb{Z}$ such that $a_i = mb_i$ for all i .

When presenting the algorithms, we use `cswap`(A, B, b) to denote a constant-time conditional swap that exchanges the values of the variables A and B if and only if $b = 1$.

3.2 Orientation

Let $K = \mathbb{Z}[\sigma]$ be an imaginary quadratic order. A K -orientation on an elliptic curve E is an injective ring homomorphism:

$$\iota : K \hookrightarrow \text{End}^0(E),$$

where $\text{End}^0(E) := \text{End}(E) \otimes_{\mathbb{Z}} \mathbb{Q}$. The pair (E, ι) is referred to as a K -oriented elliptic curve. For an imaginary quadratic order $\mathcal{O} \subseteq K$, we define ι as an $\mathcal{O}$ -orientation if the image of $\mathcal{O}$ under ι is contained within the endomorphism ring of E , i.e., $\iota(\mathcal{O}) \subseteq \text{End}(E)$. An $\mathcal{O}$ -orientation is said to be primitive if $\iota(K) \cap \text{End}(E) = \iota(\mathcal{O})$, that is, ι does not extend to an orientation by any strictly larger order $\mathcal{O}' \supset \mathcal{O}$. For any element $\alpha \in \text{End}^0(E)$, let $\text{tr}(\alpha)$ and $\text{Nrd}(\alpha)$ denote its reduced trace and reduced norm, respectively. The discriminant of α is defined as:

$$\text{Disc}(\alpha) := \text{tr}(\alpha)^2 - 4\text{Nrd}(\alpha).$$

Induced orientations. Let $\varphi : E \rightarrow E'$ be a separable isogeny of degree $\deg(\varphi)$. Given an orientation ι on E , the induced orientation $\varphi_*(\iota)$ on E' is defined by

$$\varphi_*(\iota)(\alpha) := \varphi \circ \iota(\alpha) \circ \hat{\varphi} \otimes \deg(\varphi)^{-1} \in \text{End}^0(E').$$

Two $\mathcal{O}$ -oriented curves (E, ι) and (E', ι') are *K-oriented isomorphic* if there exists an isomorphism $\psi : E \rightarrow E'$ such that $\iota' = \psi_*(\iota)$.

3.3 Class Group Action

Let $\mathcal{O} \subset K$ be an imaginary quadratic order. Let $I(\mathcal{O})$ denote the group of invertible fractional $\mathcal{O}$ -ideals, and $P(\mathcal{O})$ the subgroup of principal fractional ideals. The ideal class group is $\text{Cl}(\mathcal{O}) := I(\mathcal{O})/P(\mathcal{O})$.

For an invertible ideal $\mathfrak{a} \subset \mathcal{O}$ of norm coprime to p and an $\mathcal{O}$ -oriented curve (E, ι) over $\mathbb{F}_{p^2}$, the Deuring correspondence attaches an isogeny $\varphi_{\mathfrak{a}} : E \rightarrow E'$ with kernel

$$E[\mathfrak{a}] := \bigcap_{\alpha \in \mathfrak{a}} E[\alpha],$$

where $E[\alpha] = \{P \in E \mid \iota(\alpha)(P) = 0_E\}$. we write $E' = [\mathfrak{a}]E$ (well-defined up to isomorphism), or equivalently $\mathfrak{a} * E$ when emphasizing the action. If $\mathfrak{a} \sim \mathfrak{b}$, i.e., they are in the same class in $\text{Cl}(\mathcal{O})$, then

$$([\mathfrak{a}]E, (\varphi_{\mathfrak{a}})_*(\iota)) \cong ([\mathfrak{b}]E, (\varphi_{\mathfrak{b}})_*(\iota))$$

as K -oriented elliptic curves. Hence, $\text{Cl}(\mathcal{O})$ acts on $\mathcal{O}$ -oriented curves up to K -oriented isomorphism. When restricted to *primitive* $\mathcal{O}$ -orientations, the action is free.

3.4 Kernel Representations

In NIKE, the orientation is an essential element representing a set member. In the case that the endomorphism $\iota(\sigma)$ is a cyclic endomorphism with degree M^r and M smooth, it can be decomposed into a sequence of isogenies of degree M . Further, each isogeny can be represented by its kernel. In this specification, we define the (full) kernel representation of the orientation as follows:

Kernel representation. Let $M = \prod_{i=1}^n \ell_i^{e_i}$ with small primes ℓ_i , and suppose $E[M] \subseteq E(\mathbb{F}_{p^2})$. A *kernel representation* of the induced $\mathbb{Z}[\sigma]$ -orientation with $\text{Nrd}(\sigma) = M^r$ is a chain of cyclic isogenies

$$E = E_1 \xrightarrow{\varphi_1} E_2 \xrightarrow{\varphi_2} \cdots \xrightarrow{\varphi_r} E_{r+1} = E,$$

where $\deg(\varphi_j) = M$ and $\ker(\varphi_j) = \langle P_j \rangle$ for some $P_j \in E_j(\mathbb{F}_{p^2})$ of order M , $j = 1, 2, \dots, r$.

Full kernel representation. Let $Q_{j+1} \in E_{j+1}(\mathbb{F}_{p^2})$ generate $\ker(\widehat{\varphi}_j)$. Then the tuple

$$\mathcal{E} := \{(E_j, P_j, Q_j) \mid 1 \leq j \leq r\}$$

is called a *full kernel representation* of the induced $\mathbb{Z}[\sigma]$ -orientation.

In practice, a key property of the full kernel representation is

$$\langle P_j \rangle \cap \langle Q_j \rangle = \{0_{E_j}\} \quad \text{for all } j,$$

which, within the orientation framework, is equivalent to the condition that the relevant small primes do not ramify in the class group. In particular, this holds in common parameter regimes where $\text{Nrd}(\sigma)$ and $\text{tr}(\sigma)$ are coprime [Hou25b].

3.5 Class Group Action Tailored to NIIKE

Given a full kernel representation of the $\mathcal{O}$ -orientation $\mathcal{E}$ and a vector $(s_i)_{i=1}^n \in [0, e_i]_{i=1}^n$, one can compute each tuple of $[\prod_{i=1}^n \ell_i^{s_i}] * \mathcal{E} := \{(E'_j, P'_j, Q'_j)\}_{j=1}^r$ by the following two steps:

1. For i ranging from 1 to n , constructs the isogenies $\varphi_{j,i}^+$ with kernel $\langle [M/\ell_i^{s_i}] \varphi_{j,i-1}^+ \circ \cdots \circ \varphi_{j,1}^+ \circ \varphi_{j,0}^+(P_j) \rangle$, and $\varphi_{j,i}^-$ with kernel $\langle [M/\ell_i^{e_i-s_i}] \varphi_{j,i-1}^- \circ \cdots \circ \varphi_{j,1}^- \circ \varphi_{j,0}^-(Q_{j+1}) \rangle$, where $\varphi_{j,0}^+$ and $\varphi_{j,0}^-$ are the identity maps $\text{id}_j : E_j \rightarrow E_j$ and $\text{id}_{j+1} : E_{j+1} \rightarrow E_{j+1}$, respectively.¹
2. Let $\varphi_j^+ := \varphi_{j,n}^+ \circ \cdots \circ \varphi_{j,1}^+$, $\varphi_j^- := \varphi_{j,n}^- \circ \cdots \circ \varphi_{j,1}^-$. The curve E' is set to be the codomain of φ_j^+ (φ_j^-), and points $P'_j = \varphi_j^-(P_{j+1})$, $Q'_j = \varphi_j^+(Q_j)$.

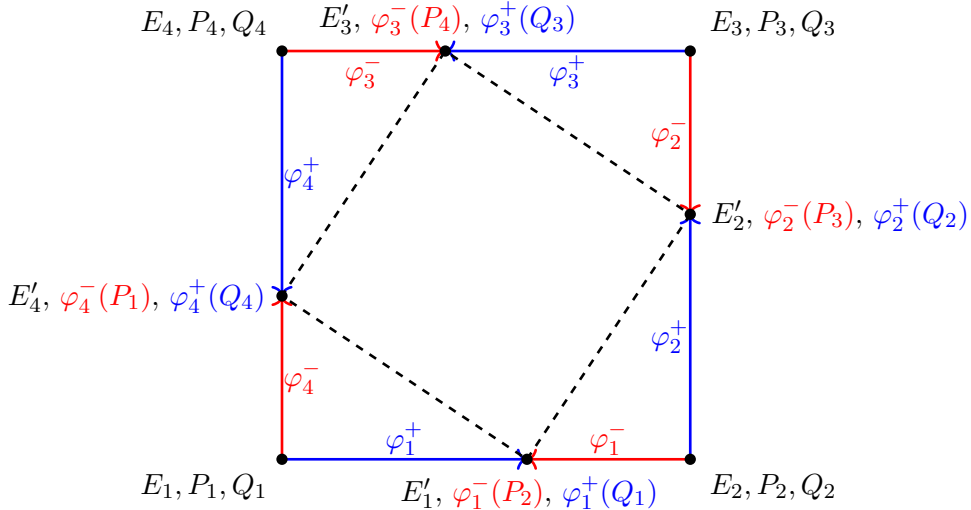


Figure 3.1: Deterministic class group action.

¹We use the convention that $Q_1 = Q_{r+1}$, $E_1 = E_{r+1}$ and $\text{id}_1 = \text{id}_{j+1}$.

GROUPACTIONOFNIIKE summarizes the above procedures. For the correctness we refer to [Hou25b]. Note that the algorithm is fully deterministic, strictly constant-time and dummy-free. Figure 3.1 illustrates the deterministic group action when $r = 4$.

Algorithm 2 **GROUPACTIONOFNIIKE**($\mathcal{E}, (s_i)_{i=1}^n$)

Input: A full kernel representation $\mathcal{E} = \{(E_j, P_j, Q_j) \mid 1 \leq j \leq r\}$ of $\mathbb{Z}[\sigma]$ -orientation on E , a vector $(s_1, s_2, \dots, s_n) \in [0, e_i]_{i=1}^n$

Output: A full kernel representation $\mathcal{E}' = \{(E'_j, P'_j, Q'_j) \mid 1 \leq j \leq r\}$ of $\mathbb{Z}[\sigma]$ -orientation on $E' = [\prod_{i=1}^n \ell_i^{s_i}] E_1$

```

1: For  $j = 1, \dots, r$  do
2:    $R_0 \leftarrow (E_j, P_j, Q_j)$ ,  $R_1 \leftarrow (E_{j+1}, Q_{j+1}, P_{j+1})$ ,  $m \leftarrow \prod_{i=1}^n \ell_i^{e_i}$ 
3:   For  $i = 1, \dots, n$  do
4:     For  $t = 1, \dots, e_i$  do
5:        $b \leftarrow t > s_i$ 
6:        $\text{cswap}(R_0, R_1, b)$ 
7:        $m \leftarrow m / \ell_i$ 
8:        $K \leftarrow \text{LADDER}(R_0[1], R_0[0], m)$ 
9:        $R_0[0], \{R_0[1], R_0[2]\} \leftarrow \text{ISOGENY}(R_0[0], \{R_0[1], R_0[2]\}, K, \ell_i)$ 
10:       $R_1[1] \leftarrow \text{LADDER}(R_1[1], R_1[0], \ell_i)$ 
11:       $\text{cswap}(R_0, R_1, b)$ 
12:    end For
13:  end For
14:   $(R_0[1], R'_j) \leftarrow (R_1[2], R_0)$ 
15: end For
16: Return  $\mathcal{E}' = (R'_1, \dots, R'_r)$ 

```

Remark 3.5.1. [Isomorphism Test] It is easy to see that the codomains of φ_j^+ and φ_j^- are isomorphic. However, in computations it may happen that the codomains of φ_j^+ and φ_j^- ($j = 1, 2, \dots, r$) do not have the same coefficients, differing by an isomorphism. In this case, an isomorphism is required to identify the codomain. In the implementation, one can choose a full kernel representation of the orientation such that only E'_r needs to be identified.

3.6 Tail Pruning

As we will see in Section 4.3, in some specific cases we do not need the full information of $\mathcal{E}'$, but only the first several tuples of it. Note that the computation for each tuple $R'_j = \{E'_j, P'_j, Q'_j\}$ in $\mathcal{E}'$ can be determined solely by two tuples $R_j = \{E_j, P_j, Q_j\}$ and $R'_{j+1} = \{E_{j+1}, P_{j+1}, Q_{j+1}\}$ in $\mathcal{E}$. Therefore, the input also does not require the full information of

$\mathcal{E}$. To summarize, **GROUPACTIONOFNIIKETAILPRUNING** illustrates the main procedures, which is also fully deterministic, strictly constant-time and dummy-free.

Algorithm 3 **GROUPACTIONOFNIIKETAILPRUNING**($\mathcal{E}, (s_i)_{i=1}^n, b$)

Input: Part of full kernel representation $\mathcal{E} = \{(E_j, P_j, Q_j) \mid 1 \leq j \leq b+1\}$ of $\mathbb{Z}[\sigma]$ -orientation on E , a vector $(s_1, s_2, \dots, s_n) \in [0, e_i]_{i=1}^n$

Output: Part of full kernel representation $\mathcal{E}' = \{(E'_j, P'_j, Q'_j) \mid 1 \leq j \leq b\}$ of $\mathbb{Z}[\sigma]$ -orientation on $E' = [\prod_{i=1}^n \ell_i^{s_i}]E_1$

```

1: For  $j = 1, \dots, b$  do
2:    $R_0 \leftarrow (E_j, P_j, Q_j), R_1 \leftarrow (E_{j+1}, Q_{j+1}, P_{j+1}), m \leftarrow \prod_{i=1}^n \ell_i^{e_i}$ 
3:   For  $i = 1, \dots, n$  do
4:     For  $t = 1, \dots, e_i$  do
5:        $b' \leftarrow t > s_i$ 
6:        $\text{cswap}(R_0, R_1, b')$ 
7:        $m \leftarrow m / \ell_i$ 
8:        $K \leftarrow \text{LADDER}(R_0[1], R_0[0], m)$ 
9:        $R_0[0], \{R_0[1], R_0[2]\} \leftarrow \text{ISOGENY}(R_0[0], \{R_0[1], R_0[2]\}, K, \ell_i)$ 
10:       $R_1[1] \leftarrow \text{LADDER}(R_1[1], R_1[0], \ell_i)$ 
11:       $\text{cswap}(R_0, R_1, b)$ 
12:    end For
13:  end For
14:   $(R_0[1], R'_j) \leftarrow (R_1[2], R_0)$ 
15: end For
16: Return  $\mathcal{E}' = (R'_1, \dots, R'_b)$ 

```

CHAPTER 4

Specification of NIIKE

This chapter introduces **Non-Interactive Isogeny-based Key Exchange**, referred to as NIIKE, including setup, key generation and key agreement and theoretical key sizes.

4.1 Setup

Given a security parameter λ , the setup of NIIKE is to generate a public parameter $\mathbf{pp} = (p, M, \{\ell_i, e_i\}_{i=1}^n, r, \iota, \sigma, E_0, \mathcal{E}_0, B)$:

- $p \in \mathbb{N}$ is a prime with $p \equiv 3 \pmod{4}$;
- $M = \prod_{i=1}^n \ell_i^{e_i} \in \mathbb{N}$ is a smooth integer divides $p^2 - 1$, where ℓ_i are small primes and $e_i \in \mathbb{N}$;
- $r \in \mathbb{N}$, which depends on M and λ ;
- (E_0, ι) is an $\mathcal{O}$ -oriented elliptic curve, where $E_0 : y^2 = x^3 + x$ is a supersingular curve defined over $\mathbb{F}_{p^2}$ with j -invariant 1728, ι is represented by the full kernel representation, and $\mathcal{O} = \mathbb{Z}[\sigma]$ satisfying $\text{tr}(\sigma) < 4(\text{Nrd}(\sigma) - \sqrt{\text{Nrd}(\sigma)})$ and $\text{Nrd}(\sigma) = M^r$;
- $\mathcal{E}_0 = \{(E_j, P_j, Q_j) \mid 1 \leq j \leq r\}$ is a full kernel representation of the $\mathcal{O}$ -orientation, which is called the distinguished origin;
- $B \in \mathbb{N}$ is the smallest integer such that the size of the private key space

$$\#\{(s_i)_{i=1}^n \mid s_i \in [0, Be_i]\} = \prod_{i=1}^n (1 + Be_i) \geq 2^{2\lambda}. \quad (4.1)$$

4.2 Key Generation

The key generation algorithm **NIIKE.KEYGEN** outputs a secret key $\mathbf{sk} := (\mathbf{sk}_i)_{i=1}^n$ and the corresponding public key $\mathbf{pk} := \mathcal{E} = \{(E_j, P_j, Q_j) \mid 1 \leq j \leq r\}$.

The key generation algorithm first selects a tuple $\mathbf{sk} = (\mathbf{sk}_i)_{i=1}^n \in [0, Be_i]_{i=1}^n$ uniformly at random and sets it as the secret key. It then applies **GROUPACTIONOFNIIKE** B times

with respect to the secret key. In each iteration, it computes $(s'_i)_{i=1}^n$ with respect to $\mathbf{sk}$: set $s'_k = e_k$ if $s_k = \mathbf{sk}_k - j \cdot e_k \geq e_k$, and $s'_k = \min(s_k, 0)$ otherwise. After that, it applies `GROUPACTIONOFNIIKE` with the input tuple $(s'_i)_{i=1}^n$. Essentially, the algorithm generates the public key $\mathbf{pk} = \mathcal{E} = \{(E_j, P_j, Q_j) \mid 1 \leq j \leq r\}$, which is a full kernel representation $\mathcal{E}' = \{(E'_j, P'_j, Q'_j) \mid 1 \leq j \leq r\}$ of the $\mathbb{Z}[\sigma]$ -orientation on $E' = [\prod_{i=1}^n \mathfrak{t}_i^{\mathbf{sk}_i}]E$.

Algorithm 4 NIIKE.KEYGEN()

Output: Secret key $\mathbf{sk} := (\mathbf{sk}_i)_{i=1}^n$ and public key $\mathbf{pk} = \mathcal{E} := \{(E_j, P_j, Q_j) \mid 1 \leq j \leq r\}$

- 1: Select an independent uniformly random tuple $\mathbf{sk} = (\mathbf{sk}_i)_{i=1}^n$ in $[0, B e_i]_{i=1}^n$
 - 2: $\mathcal{E} \leftarrow \mathcal{E}_0, (s_i)_{i=1}^n \leftarrow \mathbf{sk}$
 - 3: **For** $j = 1, \dots, B$ **do**
 - 4: **For** $k = 1, \dots, n$ **do**
 - 5: $s_k \leftarrow s_k - e_k$
 - 6: $s'_k \leftarrow \max(\min(s_k, e_k), 0)$
 - 7: **end For**
 - 8: $\mathcal{E} \leftarrow \text{GROUPACTIONOFNIIKE}(\mathcal{E}, (s'_i)_{i=1}^n)$
 - 9: **end For**
 - 10: $\mathbf{pk} \leftarrow \mathcal{E}$
 - 11: **Return** $\mathbf{sk}$ and $\mathbf{pk}$
-

4.3 Key Agreement

The primary purpose of key agreement is to compute the shared secret. In the key agreement phase NIIKE, given the other party's public key and the secret key, one can derive the shared secret $\mathbf{ss} := j(E)$, which is the j -invariant of a supersingular elliptic curve.

The key agreement algorithm shares certain similarities with the key generation algorithm: both apply the ideal class group action B times. However, whereas the output of the key generation algorithm is a full kernel representation of an orientation, the key agreement algorithm outputs only the j -invariant of a curve, namely an element of $\mathbb{F}_{p^2}$. Thus, the "full" computation of `GROUPACTIONOFNIIKE` is not always necessary in the context of key agreement. More specifically, `GROUPACTIONOFNIIKE` computes the class group action by updating the complete endomorphism information, thereby enabling the construction of the entire endomorphism data for the subsequent class group action, whereas in key agreement only one curve from the full kernel representation is required.

Recall that in `GROUPACTIONOFNIIKETAILPRUNING`, we exploit the fact that the computation of $\{E'_j, P'_j, Q'_j\}$ in $\mathcal{E}'$ is determined solely by $\{E_j, P_j, Q_j\}$ and $\{E_{j+1}, P_{j+1}, Q_{j+1}\}$ in $\mathcal{E}$, thereby avoiding the final several updates of the full kernel representation. Therefore, in the last iteration of the key agreement phase, it suffices to update only the first two tuples, $\{E_1, P_1, Q_1\}$ and $\{E_2, P_2, Q_2\}$, obtained from the penultimate iteration; accordingly,

in the penultimate iteration, it suffices to update only the first three tuples; and so forth. Recursively, at the $(B - r + 1)$ -th iteration, the complete information of the full kernel representation of the orientation is required. In other words, the $(B - r + 1)$ -th iteration is precisely the step at which the pruning process can begin.

As a result, we present **NIIKE.KEYAGR** to compute the shared secret. Note that we use **GROUPACTIONOFNIIKETAILPRUNING** instead of **GROUPACTIONOFNIIKE** for the final $r - 1$ iterations to avoid redundant computations. For an integer $b > B - r + 1$, in the b -th iteration, the update of the last $(b + r - B - 1)$ tuples is skipped. Consequently, we eliminate a total of $\sum_{t=1}^r (t - 1) = r(r - 1)/2$ tuple updates in the key agreement phase. Notably, the pruning process depends only on public iteration indices and thus preserves the deterministic, dummy-free, and constant-time features of [Hou25b].

Algorithm 5 NIIKE.KEYAGR($\mathcal{E}, \text{sk}$)

Input: Secret key $\text{sk} := (s_i)_{i=1}^n$ and the public key $\text{pk} = \mathcal{E} := \{(E_j, P_j, Q_j) \mid 1 \leq j \leq r\}$ from the other party

Output: The shared secret $\text{ss} := j(E)$

```

1:  $\mathcal{E}' \leftarrow \mathcal{E}, (s_i)_{i=1}^n \leftarrow \text{sk}$ 
2: For  $j = 1, \dots, B$  do
3:   For  $k = 1, \dots, n$  do
4:      $s_k \leftarrow s_k - e_k$ 
5:      $s'_k \leftarrow \max(\min(s_k, e_k), 0)$ 
6:   end For
7:   If  $j \leq B - r + 1$  then
8:      $\mathcal{E} \leftarrow \text{GROUPACTIONOFNIIKE}(\mathcal{E}, (s'_i)_{i=1}^n)$ 
9:   else
10:     $\mathcal{E} \leftarrow \text{GROUPACTIONOFNIIKETAILPRUNING}(\mathcal{E}, (s'_i)_{i=1}^n, B - j + 1)$ 
11:   end If
12: end For
13:  $\{(E, \_, \_)\} \leftarrow \mathcal{E}$ 
14:  $\text{ss} \leftarrow j(E)$ 
15: Return  $\text{ss}$ 

```

4.4 Theoretical Key Size

This section presents the sizes of private key, (compressed) public key and shared secret key in theory.

Private key. The private key is a vector $\text{sk} = (\text{sk}_i)_{i=1}^n \in [0, Be_i]_{i=1}^n$. Each sk_i is a non-negative integer less than Be_i . Therefore, by Equation (4.1), the private key size (in bits)

satisfies

$$\sum_{i=1}^n \log(Be_i + 1) = \log\left(\prod_{i=1}^n (1 + Be_i)\right) \geq \log(2^{2\lambda}) = 2\lambda. \quad (4.2)$$

Since B is the smallest integer such that Equation (4.1) holds, we can expect that the private key size is slightly larger than 2λ bits.

Public key. The public key is a full-kernel representation of an orientation, namely, $\mathcal{E} = \{(E_i, P_i, Q_i) \mid 1 \leq i \leq r\}$. In general, the curves are defined over $\mathbb{F}_{p^2}$, and they can be determined by their Montgomery coefficients, each of size $\log(p^2) = 2\log(p)$ bits. However, the points P_i and Q_i of order M are not defined on $E(\mathbb{F}_{p^2})$. In the implementation, we use the fact that $M \mid p^2 - 1$ to decompose a point of order M into two rational points: one is defined on $E(\mathbb{F}_{p^2})$, and the other is not defined on $E(\mathbb{F}_{p^2})$ but its x -coordinate is defined on $\mathbb{F}_{p^2}$. In the key agreement phase, only the x -coordinates of the points are required for group actions. Therefore, storing the information of P_i and Q_i requires $2 \cdot \log(p^2) = 4\log(p)$ bits. Consequently, the public key requires

$$r \times (2\log(p) + 4\log(p)) = 6r\log(p) \gtrsim 12r\lambda \quad (4.3)$$

bits of storage.

Compressed public key. One can compress the torsion basis information within the public key. Since P_i is the kernel generator of the isogeny $\varphi : E_i \rightarrow E_{i+1}$ and $\langle Q_{i+1} \rangle$ is the kernel of the dual isogeny $\widehat{\varphi}_i : E_{i+1} \rightarrow E_i$, one can use the kernel representation as the public key instead of the full kernel representation. In the key agreement phase, the other party is able to recover the full kernel representation as follows:

1. For each i , select one rational point Q'_i such that $\langle P_i, Q'_i \rangle$ is a torsion basis of $E_i[M]$;
2. Compute $Q_{i+1} = \varphi_i(Q'_i)$.

Therefore, the information of kernel representation is enough for key agreement. Additionally, from the knowledge of E_1 , the remaining curves E_i , $i = 2, \dots, r$ can be subsequently derived by computing the isogenies $\varphi_1, \varphi_2, \dots, \varphi_{r-1}$ with kernel $\langle P_1 \rangle, \dots, \langle P_{r-1} \rangle$, respectively. Therefore, it is sufficient to send $\mathbf{pk} = (E_1, P_1, \dots, P_r)$ to the other, and vice versa.

Further, one can further compress the rational point $P_i \in E_i$ by the well-known technique used in isogeny-based cryptosystems: generate a deterministic torsion basis $\langle U, V \rangle = E_i[M]$ to linearly represent P_i , and store the scalar information instead of the x -coordinates of P , that is,

$$P_i = [a_i]U + [b_i]V. \quad (4.4)$$

where $a_i, b_i \in \mathbb{Z}/M\mathbb{Z}$. Since key agreement does not need the explicit information of P_i but just a kernel generator $\langle P_i \rangle$, a_i and b_i can be compressed into a $\lceil \log(M) \rceil$ -bit scalar and

a label of n -bits. For more details we refer to [LLC⁺24]. To summarize, the compressed public key size is

$$2\log(p) + r \cdot (\lceil \log(M) \rceil + n). \quad (4.5)$$

Shared secret key. The shared secret key is the j -invariant of a supersingular elliptic curve defined over $\mathbb{F}_{p^2}$, which requires

$$\log(p^2) = 2\log(p) \gtrsim 4\lambda \quad (4.6)$$

bits.

Remark 4.4.1. To target NGCC-I and NGCC-II security levels, the prime sizes are about 2λ bits. Therefore, all the compressed key sizes are close to optimal. However, our implementation for NGCC-III security level asks for larger prime size due to efficiency. We will provide a further explanation in Chapter 5.

CHAPTER 5

Parameters

This chapter specifies the parameter constraints required by NIIKE and lists concrete parameter sets targeting NGCC security categories.

5.1 Parameter Requirements

A *parameter set* for NIIKE specifies a tuple

$$\text{pp} = (p, M, \{\ell_i, e_i\}_{i=1}^n, r, \iota, \sigma, E_0, \mathcal{E}_0, B).$$

The prime p is the characteristic of the underlying finite field. To target a λ -bit classical security level, the size of p should be $\gtrsim 2\lambda$ bits. We also limit $p \equiv 3 \pmod{4}$ such that the starting curve $E_0 : y^2 = x^3 + x$ is supersingular with endomorphism ring $\text{End}(E_0) \cong \langle 1, i, \frac{i+j}{2}, \frac{1+k}{2} \rangle$, where $i^2 = -1, j^2 = -p$ and $k = ij = -ji$.

Besides, the efficient implementation of NIIKE asks for large accessible smooth-order torsion bases. This requires that there exists a large smooth integer M dividing $p^2 - 1$, i.e., $M = \ell_1^{e_1} \ell_2^{e_2} \cdots \ell_n^{e_n}$ where ℓ_m are small primes and $e_m \in \mathbb{N}$, $m = 1, 2, \dots, n$. In Section 5.2, we will describe how to find out a suitable prime p for efficient implementation.

Next, let $\mathcal{O} \subset K = \mathbb{Z}[\sigma]$, where $\text{tr}(\sigma) < 4(\text{Nrd}(\sigma) - \sqrt{\text{Nrd}(\sigma)})$ and $\text{Nrd}(\sigma) = M^r$, and r is a positive integer depending on the security levels. The generation of σ can be done once p is given (see Section 5.3), and we store its corresponding full kernel representation $\mathcal{E}_0 := \{(E_i, P_j, Q_j) \mid 1 \leq j \leq r\}$. The generation of the full kernel representation is expensive because of the ideal-to-isogeny translation, but this part is done via precomputation in polynomial time.

Finally, we fix a positive integer B . As shown in Equation (4.1), the integer B can be easily determined once e_i and n are fixed.

The computational efficiency bottleneck of NIIKE lies in the computation of M -isogenies, including both isogeny construction and evaluation. From the above analysis, it is clear that the prime p determines the smoothness of M . Moreover, to reduce the size of B , it is desirable for M to admit a factorization into as many prime factors as possible, i.e., n is desired to be as large as possible.

5.2 Characteristic Finding

In this section, we discuss how to search for suitable parameters p for NGCC-I and NGCC-II security levels, namely, a prime of size $\approx 2\lambda$ bits such that $M \mid p^2 - 1$, where M is smooth and has a considerable number of prime factors. The core of this method is to find large smooth twins $(r, r + 1)$ —pairs of consecutive integers whose product is smooth—and then derive a suitable prime p from them.

1. **Small prime product preconstruction:** Select the first k smallest primes, e.g., 2, 3, 5, 7, 11, 13, $\dots$, and compute their cumulative product $P = \prod_{i=1}^k \ell_i$ until P exceeds the bit-length requirement of the target prime p (e.g., ≈ 256 bits for NGCC-I security levels and ≈ 512 bits for NGCC-II security levels).
2. **Random product partitioning:** Randomly partition the above small primes into two disjoint subsets S and T , such that the subset products $|S| = \prod_{\ell_i \in S} \ell_i$ and $|T| = \prod_{\ell_j \in T} \ell_j$ are both close to $\sqrt{p}$, i.e., $|S| \approx |T| \approx \sqrt{p}$.
3. **Extended Euclidean algorithm solution:** Apply the extended Euclidean algorithm to $|S|$ and $|T|$ to obtain integers s, t satisfying Bézout's identity:

$$s \cdot |S| + t \cdot |T| = \gcd(|S|, |T|) = 1.$$

Since $|S|$ and $|T|$ are products of distinct small primes, they are inherently coprime, so Bézout's identity always holds.

4. **Candidate prime generation:** Take the candidate value as $p' = 2 \cdot |s \cdot |S|| \pm 1$ (where $|\cdot|$ denotes the absolute value operation to avoid invalid candidates when s is negative).
5. **Primality validation:** Perform the Miller-Rabin pseudoprimality test on p' . If it passes the test, accept it as the final parameter p . Otherwise, adjust the subset partition or replace the small prime set, and repeat steps 2-4 until a qualified prime is found.

However, for the NGCC-III security level, strictly adhering to a prime p of size $\approx 2\lambda$ bits introduces severe theoretical and practical bottlenecks. Therefore, we deviate from this baseline and opt for a substantially larger prime p , driven by the following three core considerations:

First, as p scales, finding a sufficiently large smooth number M such that $M \mid p^2 - 1$ becomes more difficult. In fact, identifying such smooth numbers remains a prominent

open challenge, making it highly impractical to locate an optimal p that balances both security and efficiency under the NGCC-III security level.

Second, when $p \approx 2^{2\lambda}$, computing the kernel representation corresponding to a given orientation is exceedingly time-consuming. Based on our experimental evaluation, it would require more than a year of continuous precomputation to derive the required kernel representation.

Third, under this parameter size, to guarantee a sufficiently large private key space, the parameter B for the NGCC-III setting must scale up to 207. This implies that both key generation and key agreement would require updating the orientation's kernel representation 207 times, which is computationally prohibitive.

Consequently, we abandon the pursuit of such a parameter p for the NGCC-III security level, and instead employ a larger prime (≈ 16384 bits) that seamlessly satisfies the corresponding security requirements while ensuring computational feasibility, i.e., there exists an orientation whose kernel representation is available with efficient precomputation.

5.3 Orientation Finding

In the setup phase of NIKE, one needs to compute an orientation σ on $E_0 : y^2 = x^3 + x$ with known endomorphism ring $\text{End}(E) \cong \mathcal{O}_0 = \mathbb{Z}\langle 1, i, \frac{i+j}{2}, \frac{1+k}{2} \rangle$. In addition, the quaternion corresponding to the orientation should have relatively small trace and fixed reduced norm M^r . Given the knowledge of the endomorphism ring, this amounts to solving the norm equation

$$\text{Nrd}(\sigma) = N\left(a + bi + c\frac{i+\pi}{2} + d\frac{1+i\pi}{2}\right) = M^r,$$

where $a, b, c, d \in \mathbb{Z}$. After the linear change of variables $x := 2a + d$, $y := 2b + c$, we can rewrite as

$$x^2 + y^2 + p(c^2 + d^2) = 4M^r.$$

where $\text{tr}(\sigma) = x$. For security, we additionally require that $|\text{Disc}(\sigma)| = 4\text{Nrd}(\sigma) - \text{tr}(\sigma)^2 = 4M^r - x^2$ be either

- prime, or
- the product of two large, distinct primes.

Note that the second case is the best we can hope for when $r = 2s$ is even, in view of the factorization

$$4M^{2s} - x^2 = (2M^s + x)(2M^s - x).$$

Although this requirement on the discriminant is not strictly necessary, we explain in Chapter 9 why such a discriminant is desirable, which implies that any $\mathbb{Z}[\sigma]$ -orientation is automatically primitive.

For our purposes, we define the following algorithms:

- `nextprime( $x$ )`: Given an integer x , outputs the smallest prime q such that $q > x$;
- `isprime( $x$ )`: Given an integer x , outputs `true` if it is a prime, or `false` otherwise;
- `isprime( $x, y$ )`: Given integers x, y , outputs $z \in \mathbb{N}$ such that $z^2 \equiv x \pmod{y}$.

The finding procedures are summarized in [EndomorphismGeneration](#).

Algorithm 6 ENDOMORPHISMGGENERATION(p, r, M)

Input: The base field characteristic p , integers r, M such that $M^r > p$

Output: An quaternion σ with reduced norm M^r and relatively small prime trace

```

1:  $N \leftarrow M^r, t = 1, a = 1$ 
2: while true do
3:   while true do
4:      $a \leftarrow \text{nextprime}(a), f \leftarrow 4 \cdot M^r - a^2$ 
5:     If  $r \bmod 2 \equiv 1$  then
6:       If isprime( $f$ ) then
7:         break
8:       end If
9:     else
10:       $f_1 \leftarrow 2 \cdot M^{r/2} + a, f_2 \leftarrow 2 \cdot M^{r/2} - a$ 
11:      If isprime( $f_1$ ) and isprime( $f_2$ ) then
12:        break
13:      end If
14:    end If
15:     $D \leftarrow (4 \cdot M^r - a^2) \bmod p$ 
16:    If issquare( $D, p$ ) then
17:       $y \leftarrow \text{sqrt}(D, p)$ 
18:      If  $y \bmod 2 \equiv 1$  then
19:         $y \leftarrow p - y$ 
20:      end If
21:       $m \leftarrow \frac{4 \cdot M^r - a^2 - b^2}{p}$ 
22:       $c, d, \text{found} \leftarrow \text{CORNACCHIA}(m)$ 
23:      If found then
24:         $\sigma \leftarrow \frac{a-c}{2} + \frac{b-d}{2}i + cj + dk$ 
25:        break
26:      end If
27:    end If
28: Return  $\sigma$ 

```

Regarding the NGCC-III security level, [EndomorphismGeneration](#) can still determine a required orientation. However, as mentioned in Section 5.2, computing its corresponding kernel representation is difficult due to computationally expensive procedures, which will

be detailed in Section 5.4.

Therefore, instead of finding an orientation using a small prime of approximately 2λ bits via [EndomorphismGeneration](#), we restrict $p \equiv 7 \pmod 8$ and set the orientation to $\sigma = \pi - 1$, where π is the Frobenius map of the curve $E_0 : y^2 = x^3 + x$. According to our security analysis in Chapter 9, $|\text{Disc}(\sigma)| \approx 2^{16384}$, which implies that $p \approx 2^{16384}$. The construction of the kernel representation is deferred to Section 5.4.

5.4 Kernel Representation Construction

Before introducing the construction of the kernel representation, we briefly review our setting. In the public-parameter generation procedure, one first constructs a primitive K -orientation of E_0 , where $K = \mathbb{Z}[\sigma]$ is an imaginary quadratic order and $\sigma \in \text{End}(E_0)$ is an explicitly given endomorphism satisfying $\text{Nrd}(\sigma) = M^r$. Here $M \mid (p+1)$ is chosen to be smooth, so that $E_0[M] \subset E(\mathbb{F}_q)$. This property ensures that the kernel of each cyclic M -isogeny appearing in the representation is generated by an $\mathbb{F}_q$ -rational point, and therefore the corresponding isogenies can be evaluated using elliptic-curve arithmetic over the base field and Vélu's formula. For the curve $E_0 : y^2 = x^3 + x$, we know its full endomorphism ring (see Remark 2.4.1) and thus its orientation is easy to be sampled (as we have done in Section 5.3). After obtaining such an orientation, we need to compute its kernel representation. This constitutes the final, and most computationally expensive, step of our setup phase. The purpose of a kernel representation is to express the endomorphism σ as a composition of r cyclic isogenies, each of degree M . More precisely, one constructs a closed chain of isogenies $E_0 = E_1 \xrightarrow{\varphi_1} E_2 \xrightarrow{\varphi_2} \dots \xrightarrow{\varphi_{r-1}} E_r \xrightarrow{\varphi_r} E_{r+1} = E_0$, where each $\varphi_j : E_j \rightarrow E_{j+1}$ is a cyclic isogeny with kernel generated by a point $P_j \in E_j(\mathbb{F}_q)$ of order M , namely $\ker \varphi_j = \langle P_j \rangle$. If, in addition, one is given points $Q_j \in E_j(\mathbb{F}_q)$ such that $\langle Q_j \rangle$ is the kernel of the dual of the previous isogeny, i.e., $\langle Q_j \rangle = \ker \widehat{\varphi}_{j-1}$, with the convention $\varphi_0 = \varphi_r$, then the data $\{(E_j, P_j, Q_j) \mid 1 \leq j \leq r\}$ is called a full kernel representation.

First, we have already obtained an explicit endomorphism σ by solving a norm equation. Concretely, σ is written as an integral linear combination of the basis elements of $\text{End}(E_0)$, namely $\sigma = a + bi + c(i + \pi)/2 + d(1 + i\pi)/2$, where $a, b, c, d \in \mathbb{Z}$, and it satisfies $\text{Nrd}(\sigma) = M^r$. Therefore, the problem now changes from finding a large-norm endomorphism to converting this explicit endomorphism into a computable isogeny chain. This conversion is essential for the construction, since representing an orientation directly by an endomorphism of degree M^r would lead to a kernel that is too large to manipulate efficiently, whereas decomposing the endomorphism into r cyclic M -isogenies allows every step to be carried out on the M -torsion. The main idea is thus to represent an endomorphism of arbitrarily large degree by a chain of smooth-degree isogenies, thereby increasing the discriminant of the orientation without increasing the size of the base field.

We now describe the construction. Let $O = \mathbb{Z}[\sigma]$ and write $M = \prod_i \ell_i^{e_i}$. The class group action framework assumes that $\gcd(M, \text{Disc}(O)) = 1$, equivalently $\gcd(\text{Nrd}(\sigma), \text{tr}(\sigma)) = 1$.

Under this assumption, the principal $\mathcal{O}$ -ideal generated by σ factors as $(\sigma) = \prod_i (\ell_i, \sigma)^{re_i} = \mathfrak{e}^r$, where $\mathfrak{e} = \prod_i (\ell_i, \sigma)^{e_i} = (M, \sigma)$. Each ideal (ℓ_i, σ) is an invertible $\mathcal{O}$ -ideal, and the ideal $\mathfrak{e}$ corresponds to one horizontal isogeny of degree M . Consequently, if the current curve is E_j , the desired kernel is $E_j[M, \sigma] = E_j[\mathfrak{e}]$, and it should be generated by a point P_j of order M .

The first step is to construct the point P_1 on the base curve $E_1 = E_0$. Since $\langle P_1 \rangle$ should be equal to $E_1[M, \sigma]$, one can proceed by sampling points in $E_1[M]$. Let $R \in E_0[M]$, and compute $S = \hat{\sigma}(R)$, where $\hat{\sigma}$ denotes the dual endomorphism of σ . In the chosen basis of $\text{End}(E_0)$, the dual endomorphism corresponds to quaternion conjugation. Hence, once the coefficients a, b, c, d are known, $\hat{\sigma}$ can be explicitly evaluated on points of $E_1(\mathbb{F}_q)$. Since $\sigma\hat{\sigma} = [\text{Nrd}(\sigma)] = [M^r]$, every $R \in E[M]$ satisfies $\sigma(\hat{\sigma}(R)) = [M^r]R = 0_{E_0}$. Also, $[M]\hat{\sigma}(R) = \hat{\sigma}([M]R) = 0_{E_0}$. Hence $\hat{\sigma}(R) \in E_0[M, \sigma]$. By repeatedly sampling random points $R_i \in E_0[M]$, computing $S_i = \hat{\sigma}(R_i)$, and checking the least common multiple of the orders of the S_i , one obtains enough images to span $E_0[M, \sigma]$. Once $\text{lcm}_i(\text{ord}(S_i)) = M$, the sampled images generate the full cyclic subgroup $E_0[M, \sigma]$, and a generator P_1 of order M can be formed as a suitable linear combination of the S_i . The paper explicitly uses this random sampling and image-computation method to obtain P_1 .

The construction of this generator may be carried out componentwise over the prime-power factors of M . Suppose $M = \prod_i \ell_i^{e_i}$. For each factor $\ell_i^{e_i}$, choose an image point S_{k_i} whose order contains the full $\ell_i^{e_i}$ -part, and set $P_{1,i} = [M/\ell_i^{e_i}]S_{k_i}$. Then $P_{1,i}$ lies in the ℓ_i -primary component and has the required $\ell_i^{e_i}$ -order. Taking $P_1 = \sum_i P_{1,i}$ gives a point of order M with $\langle P_1 \rangle = E_1[M, \sigma]$. This procedure uses the Chinese remainder decomposition of the M -torsion to combine several sampled image points into one cyclic kernel generator. For a full kernel representation, one also needs an initial point Q_1 generating the dual direction. This is obtained analogously: sample random points $R'_i \in E_0[M]$, compute $T_i = \sigma(R'_i)$, and use the fact that $\hat{\sigma}\sigma = [M^r]$ to conclude that $T_i \in E[M, \hat{\sigma}]$. Applying the same order test and prime-power recombination gives a point Q_1 such that $\langle Q_1 \rangle = E_0[M, \hat{\sigma}]$.

Once P_1 has been obtained, the first isogeny is defined as $\varphi_1 : E_1 \rightarrow E_2 = E_1/\langle P_1 \rangle$. Since $\langle P_1 \rangle = E_1[M, \sigma]$, this isogeny corresponds to the action of the ideal $\mathfrak{e} = (M, \sigma)$. We then push Q_1 forward along φ_1 and define $Q_2 = \varphi_1(Q_1)$. From [Hou25b, Proposition 2.1], under the condition that $\text{Nrd}(\sigma)$ is coprime to the discriminant, one has $E_0[\sigma] \cap E_0[\hat{\sigma}] = \{0_{E_0}\}$, and the action of the ideal (M, σ) pushes the $\hat{\sigma}$ -kernel to the corresponding kernel on the codomain curve. Therefore, φ_1 preserves the order of Q_1 , and Q_2 generates $E_2[M, \hat{\sigma}]$, which is precisely the kernel of the dual isogeny $\hat{\varphi}_1$. This property allows all subsequent points Q_j to be obtained by pushing forward along the isogeny chain.

The next task is to find P_2 , a generator of $E_2[M, \sigma]$. In principle, if one had an isogeny $\psi : E_1 \rightarrow E_2$ corresponding to an ideal $I_\psi \sim I_1 = (M, \sigma)$ with norm coprime to M , then $\psi(P_1)$ would generate $E_2[M, \sigma]$. The construction uses the $\mathcal{O}$ -ideal $I = (M^{r-1}, \hat{\sigma}) = \bar{\mathfrak{e}}^{r-1}$. Since $(\sigma) = \mathfrak{e}^r$, one has $[\mathfrak{e}]^r = 1$ in $\text{Cl}(\mathcal{O})$, and since $[\bar{\mathfrak{e}}] = [\mathfrak{e}]^{-1}$, it follows that $[I] = [\bar{\mathfrak{e}}]^{r-1} = [\mathfrak{e}]$. Thus I corresponds to a horizontal isogeny $\psi = \varphi_I : E_1 \rightarrow E_2$ of degree M^{r-1} , whose kernel is $E_1[M^{r-1}, \hat{\sigma}]$. Directly computing this kernel can be expensive, especially when

$r > 2$, because a generator of $\ker \psi$ may only be defined over a large extension of $\mathbb{F}_q$.

To reduce the computational costs, we use the KLPT algorithm to solve this problem. The KLPT algorithm is used to find a left $\mathcal{O}_0$ -ideal I_ψ equivalent to I_1 , chosen with a pleasant norm. In this context, pleasant norm means two things: first, $n(I_\psi)$ is coprime to M ; second, translating I_ψ to an isogeny φ_{I_ψ} through the Deuring correspondence is computationally feasible, in the sense that its kernel is generated by torsion points of relatively small order defined over relatively small extensions of the base field.

The key relation produced by this replacement is as follows. Since I_ψ is equivalent to I_1 , there exists an element $\beta \in \mathcal{O}_0$ such that $\overline{I_\psi} I_1 = \mathcal{O}_0 \beta$. At the level of isogenies, this gives an endomorphism $\beta = \widehat{\psi} \circ \varphi_1 \in \text{End}(E_1)$. Applying ψ to both sides yields $[n(I_\psi)] \circ \varphi_{I_1} = \psi \circ \beta$. Since $n(I_\psi)$ is coprime to M , the multiplication map $[n(I_\psi)]$ is an automorphism on the M -torsion. Therefore, the point $P_2 = (\psi \circ \beta)(P_1)$ differs from the direct image $\varphi_1(P_1)$ only by an invertible scalar on $E_1[M]$, and the two points generate the same cyclic subgroup of order M . It follows that $\langle P_2 \rangle = E_2[M, \sigma]$.

After P_2 has been found, one computes $E_3 = E_2 / \langle P_2 \rangle$. The same procedure is then repeated. In general, once E_{j+1} has been constructed, one obtains P_{j+1} by taking $I_j = (M^j, \sigma) = \mathfrak{e}^j$. This ideal corresponds to a horizontal isogeny from E_1 to E_{j+1} . The KLPT algorithm can find an equivalent left $\mathcal{O}_0$ -ideal I_ψ and an element $\beta \in \mathcal{O}_0$ such that $\overline{I_\psi} I_j = \mathcal{O}_0 \beta$, with $n(I_\psi)$ coprime to M , and define $P_{j+1} = (\psi \circ \beta)(P_1)$. Since multiplication by $n(I_\psi)$ is invertible on the M -torsion, this point generates $E_{j+1}[M, \sigma]$. Repeating this construction yields $P_1, \dots, P_r$ and the curves $E_1, \dots, E_r$. The final isogeny $E_r \rightarrow E_r / \langle P_r \rangle$ returns to a curve isomorphic to E_1 , because the whole chain corresponds to $\mathfrak{e}^r = (\sigma)$, and the action of a principal ideal gives a closed cycle in the oriented setting.

The recovery of the dual-kernel points Q_j is more direct. The initial point Q_1 has already been obtained by random sampling and evaluation under σ , and it generates $E_1[M, \widehat{\sigma}]$. After computing each isogeny $\varphi_j : E_j \rightarrow E_{j+1}$, define $Q_{j+1} = \varphi_j(Q_j)$. Since $E_0[\sigma] \cap E_0[\widehat{\sigma}] = \{0_{E_0}\}$, the kernel $\langle P_j \rangle = E_j[M, \sigma]$ of φ_j is disjoint from $\langle Q_j \rangle = E_j[M, \widehat{\sigma}]$, so the order of Q_j is preserved under the push-forward. Moreover, since $\varphi_j(E_j[M, \widehat{\sigma}]) = E_{j+1}[M, \widehat{\sigma}]$, the point Q_{j+1} generates $E_{j+1}[M, \widehat{\sigma}]$, which is the kernel of the dual isogeny $\widehat{\varphi}_j$. Consequently, the data $\{(E_j, P_j, Q_j) \mid 1 \leq j \leq r\}$ is the desired full kernel representation.

Algorithmically, the construction can be summarized as follows. The input consists of the base curve $E_0 : y^2 = x^3 + x$, a smooth integer $M \mid p^2 - 1$, an integer r , and an explicit endomorphism $\sigma = a + bi + c(i + \pi)/2 + d(1 + i\pi)/2$ satisfying $\text{Nrd}(\sigma) = M^r$. First, sample random points $R_i \in E_0[M]$, compute $\widehat{\sigma}(R_i)$, and continue until the least common multiple of the orders of these images is M ; then combine the images to obtain $P_1 \in E[M, \sigma]$. Second, sample random points $R'_i \in E[M]$, compute $\sigma(R'_i)$, and use the same procedure to obtain $Q_1 \in E[M, \widehat{\sigma}]$. Third, compute $E_2 = E_1 / \langle P_1 \rangle$ and set $Q_2 = \varphi_1(Q_1)$. Fourth, for $j = 1, \dots, r - 1$, take $I_j = (M^{r-j}, \sigma)$, use the KLPT algorithm to obtain an equivalent ideal I_ψ and an element β , compute $P_{j+1} = (\psi \circ \beta)(P_1)$, and then set $E_{j+2} = E_{j+1} / \langle P_{j+1} \rangle$. Finally, push Q_1 along the resulting isogeny chain to obtain all points Q_j .

The correctness of the construction follows from three facts. First, the initial random

images lie in the desired kernels. For every $R \in E[M]$, the point $\hat{\sigma}(R)$ is killed by both $[M]$ and σ , hence belongs to $E[M, \sigma]$; similarly, $\sigma(R)$ belongs to $E[M, \hat{\sigma}]$. Once the sampled images cover the full prime-power components of the cyclic group, their recombination gives a generator of the entire kernel. Second, the ideals $I_j = (M^j, \sigma)$ used in the recursive step represent the correct class-group action: they send E_1 to E_j , and therefore the image of P_1 under the corresponding isogeny generates $E_j[M, \sigma]$. Replacing I_j by an equivalent KLPT ideal changes the image only by multiplication by an integer coprime to M , which is invertible on the M -torsion, so the generated cyclic subgroup is unchanged. Third, the points Q_j are obtained by the natural push-forward of the dual kernels. Since the σ -kernel and the $\hat{\sigma}$ -kernel intersect trivially, the order and kernel interpretation of each Q_j are preserved at every step. Hence the output gives both the decomposition of σ into a chain of cyclic M -isogenies and the dual-kernel information required for the full representation.

In the overall construction, the role of a kernel representation goes beyond merely evaluating an endomorphism. It converts an orientation given by an endomorphism of degree M^r and very large discriminant into r executable M -isogeny steps over the base field. Since M is composed of small prime-power factors, each M -isogeny can itself be decomposed into a chain of ℓ_i -isogenies, and the entire orientation can therefore be represented and evaluated as a smooth isogeny chain over $\mathbb{F}_q$. In terms of computational cost, the random sampling of M -torsion points and the evaluation of σ and $\hat{\sigma}$ are relatively lightweight. The dominant precomputation cost lies in the effective Deuring correspondence and the KLPT steps, whose purpose is to replace high-degree ideal isogenies by equivalent isogenies that can be computed in polynomial time.

In the NGCC-I and NGCC-II security level settings, we successfully computed a kernel representation of the orientations. Unfortunately, this approach becomes prohibitively time-consuming for the NGCC-III setting. As Section 5.2 noted, we estimate that generating the kernel representation corresponding to the NGCC-III orientation would require well over a year of computation. This bottleneck stems primarily from two factors: first, at this security level, M comprises a large number of prime factors, making it exceptionally difficult to find torsion subgroups on E of orders coprime to M within a relatively small extension field; second, the increased size of p significantly slows down the transformation process of the ideal I_ψ .

The primary computational bottleneck of the construction lies in the heavy ideal-to-isogeny translation steps. To circumvent this overhead, we strategically restrict r to completely eliminating the need for recursive ideal-to-isogeny translations. Furthermore, we ensure the existence of an explicit, easily evaluable endomorphism $\sigma = \pi - 1$, where π is the Frobenius endomorphism. Consequently, the generation of the kernel representation simplifies drastically to computing only a single pair of points, P_1 and Q_1 . This process becomes highly efficient, requiring only standard random point sampling, which bypasses the costly precomputations entirely. Although there is no theoretical limitation preventing us from including the power of 2, this leads to some practical technicalities when using Kummer arithmetic for Montgomery curves. Same as [Hou25a, Algorithm 3],

we choose P_1 and Q_1 to be rational points of maximal odd order, and finally compute the 4-isogeny to check the isomorphism property. As a result, the class group action and the key exchange algorithms should be modified correspondingly. We illustrate the modifications in [MODIFIEDGROUPACTIONOFNIIKE](#), [NIIKE.MODIFIEDKEYGEN](#) and [NIIKE.MODIFIEDKEYAGR](#). Note that in this case the tail pruning technique does not work for efficient key agreement performance, since $B \leq r$ in our setting.

Algorithm 7 [MODIFIEDGROUPACTIONOFNIIKE](#)($E, P, Q, (s_i)_{i=1}^n$)

Input: A supersingular elliptic curve E , points $P \in E(\mathbb{F}_p)$ and $Q \in E^t(\mathbb{F}_p)$ of maximal odd order, a vector $(s_1, s_2, \dots, s_n) \in [0, e_i]_{i=1}^n$

Output: A tuple (E', P', Q') , where $E' = [\prod_{i=1}^n \mathfrak{f}_i^{s_i}]E$ is a supersingular elliptic curve, points $P' \in E'(\mathbb{F}_p)$ and $Q' \in (E')^t(\mathbb{F}_p)$ are of maximal odd order

```

1:  $R_0 \leftarrow (E, P, Q)$ ,  $R_1 \leftarrow (E, Q, P)$ ,  $m \leftarrow \prod_{i=1}^n \ell_i^{e_i}$ 
2: For  $i = 1, \dots, n$  do
3:   For  $t = 1, \dots, e_i$  do
4:      $b \leftarrow t > s_i$ 
5:      $\text{cswap}(R_0, R_1, b)$ 
6:      $m \leftarrow m / \ell_i$ 
7:      $K \leftarrow \text{LADDER}(R_0[1], R_0[0], m)$ 
8:      $R_0[0], \{R_0[1], R_0[2]\} \leftarrow \text{ISOGENY}(R_0[0], \{R_0[1], R_0[2]\}, K, \ell_i)$ 
9:      $R_1[1] \leftarrow \text{LADDER}(R_1[1], R_1[0], \ell_i)$ 
10:     $\text{cswap}(R_0, R_1, b)$ 
11:   end For
12: end For
13:  $R_1[2] \leftarrow \text{EVALFOURISOG}(R_0[0], R_0[1])$ 
14: Return  $R_1[0], R_1[2], R_1[1]$ 

```

Algorithm 8 [NIIKE.MODIFIEDKEYGEN](#)()

Output: Secret key $\text{sk} := (\text{sk}_i)_{i=1}^n$ and public key $\text{pk} := \{E, P, Q\}$

```

1: Select an independent uniformly random tuple  $\text{sk} = (\text{sk}_i)_{i=1}^n$  in  $[0, e_i]_{i=1}^n$ 
2:  $\text{pk} \leftarrow \text{MODIFIEDGROUPACTIONOFNIIKE}(E_1, P_1, Q_1, \text{sk})$ 
3: Return  $\text{sk}$  and  $\text{pk}$ 

```

5.5 Parameter Sets

The following parameter sets target different security levels and satisfy the required security criteria. The concrete security of these parameters will be discussed in Chapter 9. We also list the non-twisted isogeny degree M^+ and the twisted isogeny degree M^- , satisfying that

$$M = M^+ M^-.$$

Algorithm 9 NIIKE.MODIFIEDKEYAGR()

Input: Secret key sk and the public key $pk = \{E', P', Q'\}$ from the other party

Output: The shared secret $ss := j(E)$

- 1: $(E, _, _) \leftarrow \text{MODIFIEDGROUPACTIONOFNIIKE}(E', P', Q', sk)$
 - 2: $ss \leftarrow j(E)$
 - 3: **Return** ss
-

NGCC-I :

$$\lambda = 128, \log_2 p = 248, r = 13, B = 35,$$

$$p = 0x53d1c0debdc0c1ba3edd760148d21a078008f642a9f26f65995c4d86fc6e$$

$$fff7$$

$$M^+ = 0x24bed52b01b8d020e25aec3b301b2fbf1d75d56d89d48d7b$$

$$= 3^4 \cdot 5 \cdot 7 \cdot 11^2 \cdot 19 \cdot 23 \cdot 29 \cdot 31 \cdot 41 \cdot 43 \cdot 47 \cdot 53 \cdot 71 \cdot 79 \cdot 83 \cdot 89 \cdot 97 \cdot 107$$

$$\cdot 109 \cdot 113 \cdot 131 \cdot 137 \cdot 149 \cdot 151 \cdot 157 \cdot 163 \cdot 167 \cdot 181 \cdot 229 \cdot 239 \cdot 269$$

$$M^- = 0x3acebf3dd1f3042710750fce762eef$$

$$= 13 \cdot 17 \cdot 37 \cdot 59 \cdot 61 \cdot 67 \cdot 73 \cdot 101 \cdot 103 \cdot 127 \cdot 139 \cdot 173 \cdot 179 \cdot 191 \cdot 193$$

$$\cdot 199 \cdot 211 \cdot 241$$

NGCC-II :

$$\lambda = 256, \log_2 p = 511, r = 14, B = 70,$$

$$p = 0x62af377936addf63a1c444dc3495ee857efbb63a67acbd8f9be1b2613d6d03b$$

$$744ca4184f9080721939ce1b31b9aebfe4581c51d2a08996047c166dc811b8e3f$$

$$M^+ = 0x111bfe3896341f28ceb842cd636de724dbafea02a82cd15fe14745256f4dd61$$

$$770651044493edb23aa5084f997322f03$$

$$= 3 \cdot 7^3 \cdot 11^2 \cdot 13 \cdot 17 \cdot 19 \cdot 23 \cdot 31 \cdot 37 \cdot 47 \cdot 53 \cdot 59 \cdot 73 \cdot 83 \cdot 97 \cdot 101 \cdot 103 \cdot 107 \cdot 113$$

$$\cdot 127 \cdot 131 \cdot 137 \cdot 139 \cdot 149 \cdot 151 \cdot 163 \cdot 167 \cdot 179 \cdot 181 \cdot 191 \cdot 223 \cdot 227 \cdot 229 \cdot 233$$

$$\cdot 239 \cdot 257 \cdot 263 \cdot 269 \cdot 281 \cdot 307 \cdot 311 \cdot 313 \cdot 317 \cdot 337 \cdot 347 \cdot 367 \cdot 373 \cdot 379 \cdot 383$$

$$\cdot 461 \cdot 593 \cdot 787 \cdot 937$$

$$M^- = 0xc3d632025aa22b59429731bc4aec11b07edf774cda63d85acd5edf9$$

$$= 5 \cdot 29 \cdot 41 \cdot 43 \cdot 61 \cdot 67 \cdot 71 \cdot 79 \cdot 89 \cdot 109 \cdot 157 \cdot 173 \cdot 193 \cdot 197 \cdot 199 \cdot 211 \cdot 241$$

$$\cdot 251 \cdot 271 \cdot 277 \cdot 283 \cdot 293 \cdot 331 \cdot 349 \cdot 353 \cdot 359 \cdot 443 \cdot 587 \cdot 601 \cdot 607$$

NGCC-III :

$$\lambda = 512, \log_2 p = 16392, r = 2, B = 1,$$

$p = 0xb101ae10306a5ae9f995c5492feb04c8cd36d5e0593342838521e5cb0ef944d$
 $0f01a2e81ed74cd76b4c75c45c071712bc3d257b5a141d8da4ac30e3581349b6a$
 $cd9eb37c3e21618353687ef936185c36f59227d6b371dfa92e874bd926f1ddaa5$
 $b6e031906918c88372cb4312ed4817b8ce1a0573fba2ba8e6d25a0a50fe8fcbda$
 $d4003e1dea3a0dc5d9c3e0e8fe733381d57198acb2feb395440ac9a5c1dd605ad$
 $4287962bca911468c028c08d5b8c685820414cbce2e5d8a3d7c30135fd167b364$
 $7429a8056cfd6f259d7db9dc50833b69d81ce3c28b8881d71bdc606b6a00123a8$
 $607c6ce21191353fcabc270613370e3993beed976fa16944fda221defd8e58235a$
 $ab286fd26fa893bd07247f447be5d93a4845a2e8999cc5b2545fd6575082ca06e$
 $30c6054adc26e6591c096806af4f1342101772a6a7ef451af0d99c48a882c9617$
 $f352ce8f444ab4b9bed613d03dba0f0abd31ef052c0089f1bdac5caeb974ef7cc$
 $45ad16fd91b5b108de929aced307ca813988e94831f70ea9d1ec159e9a3280664$
 $5b0239e9aade3616c52b6afecc56adb3cee2b8c8547dee1255b1ada323144d62c$
 $31d77c9168577c7e5ba5c88fb4285c74a03ffac3467015c3e0aa51810d57d2a73$
 $883bc0153cc4278e1c748865bf19f8e68eb1d3e738d1fe9d31e8247330136de61$
 $0514d770bdb2d495327b982650a62f66385a7842e4d9607f312aa8f00e7260e34$
 $43439cd3b02e8cabf26e3216555f838f91a4500ae1ffbaa6ee97c27700808b435$
 $7c573ac94e4959a73828a5896968f0b991ed04cb54fac07eac52fafc4237cf9d7$
 $d643b5c234d07ca69a9e2f5c1e414d5cf93a80412e7086dba91866272d44dd65a$
 $3e60eb7e11898d8c50ea4146aed17d8b1434760b21733639ba77c10843036dc92$
 $b1c3dd0123030738005797c25b4433bc951c1ab58716745555d85055ea24100d9$
 $79dcdd4664a12b6cda5c8fb6da779e4f73ee589e483ee57c8f457e1d113f188d9$
 $1231dcc5d2a652fce3e5520e1ae26151ae11ee8228247b9d512ca0e7b68c94d00$
 $d7c8f2d050b99c3c6901fa3d4695ac0bdede743abc89950e5f4f227876c285844$
 $0675b2d558b0cd571b6e73bae7e6fb8fac42631e35788c7c65989ae7e5321c06a$
 $f81d7f67e31797c6f2cd1b287ad5a3174786f9d6ff41148622f98c1d61438651b$
 $9d9721047436311ea4deb2adef18c978e7d6a42b5e50d0251e16d1475251ab158$
 $268fe5d9f21c86c3efdb037debdee9ac83b23d7c09204c021e524e19a3c9601c7$
 $d7cc818383a787574b535d0d251d15507348fb2423fe2631e6e852b65cbc5004c$
 $05b732444c0a5f07d842c051fa3c016d9eca25cb9f754455e5408519d0851eb39$
 $6bb3ac8c38bf67a67d1c631591c4961e5d7246264b257b10d6b4e1b6dd9b619a6$
 $8778eb52e2adb7e959a291a776b221195e9d845673e67066a5f0017a2b3ab32cb$

7f4e814e4f79090b7e80c0f8006b20b6d276965b481a2e3346b748b92b72fcaee
 2a6c99e2c557e4815af8b8c45a9aa4c1dc91a939ca426fe88cbbc3a7f4d1fcb9a
 8fcd8b349fefc89d62110a948a825dc6abb5438e0c825e9c898e64bac16c9c886a
 4e460463f4c12b61289496115b9a45c5e9e42ead86a0bf5f67a20581da325ec66
 db575c47c6eefd68c629a42bc1e6887d483ba3d9de7984d989cb5b3f4c1696848
 f2c5a67dc03086498a092d261ae1bd4fc22db2b4a12d13e2d3a46d339f7c7a66c
 ba55ba36abfe2c52672221e06b3d190809860e90507a5dad0b1712d95a9ba1d40
 b83ddbfe7e5711df972620e9ad947ce995153fa8eb05d8f1ee891946d8ce4a71
 e0882214fffc5bc59ab9b34f3916a22977fed901f38077550c7ec80e9f594bbcc8
 9b7e932edec2897035384d3b49fc3236197af2c32bb6e67532c40c91d1280eb16
 433744c4d6cec0a45bb512fe029aeda29e968d7384daa33764ec4e77106c568be
 1e6d081ef882c60719eb61a0f8262d5378b9f697e672c5a01fcec12ec0d577e13
 e842ecf05ca00d12cbba18e433d42a2d508105247f1c85abd541e4bc542c500aa
 7efd6f9844ffefffdde83b16c8c724845040749817bcec4d1fcce463f6ec908da
 eadff7cf4ded296eb7f4e3af2c73edf9318f842f3ba7e1f23722001fb9f482410
 02cbf93ea984a99916ca93d5bc04fff5b544c6d349999a5bc5ff61344b190fbb4
 5f930a2d4dcb226b3d4466866cce37e0f24bb46a945ccee60b63c0c4195609346
 221953df23b1860a656dc67cd8792be1ce1f5700aa3bcf063c43ac4fc96cee29c
 4d3156bf2e567ee225036e16f24e763e4eb39a506b66c12b260e50d443b410289
 bf3ca1d5bd28b04e58e78b54b11470e004f39b16157ed2b19497ded9c90744fc6
 ae5e4790e3ee9b4c2c865c80994cade0348e6b8f18f209ba30277416a9c511378
 e16ab8ea8f5473c14e7e4a677133742520b13cd80a52cc37569d32c56cf548b57
 948a650e97e9d1dc559854356ae18787b29e99337462c4cf1ef2e9530547b620c
 6e5d57d80559cda2af9972603c316e9b95354c672acb5d16e0dc912eb9d2357b6
 5721da59be7b9a6d405b04bb179a7ddc0a9d3bc00d69bd6625d4b3834ec409d45
 ddea6f6ebc62a6b58652f069a5537a31720d0de153d01a4df07d6222a786f364d
 37f73e81e27990ca8c283949ac73a4e6e9eed8b8c0c0baa6ac682aed68a84624e
 baa877846073080b0b82f0cae8618e518a9848690c3f0eb3efb02f395be6bee50
 914cee317da268762ca3df653f1e3f5448742896b795c29b178ca2b281174ac8b
 35615f9990dd36054c4c50dcee2fa3d3c43f02430eade3abbd51f6bd2afa0bac9
 3368ffa4ce4ecd86083595c41e149406afbfedf06dd211cdced1f84998f3035c7
 88e8b

$$M^+ = p+1$$

$$M^- = 1$$

CHAPTER 6

Test Vectors

The 'Test Vectors' folder within the submitted media file contains test vector files specific to NIIKE parameters, corresponding to the security levels of NGCC-I, NGCC-II, and NGCC-III. Each file encompasses values for private keys (**sk**), public keys (**pk**), shared secret keys (**ss**) and seeds (**seed**).

CHAPTER 7

Performance analysis

We submit three code packages: a reference implementation written in ISO C, an optimized implementation targeting Intel Broadwell with finite field arithmetic implemented in x86 assembly, and an additional implementation utilizing the AVX-512IFMA instruction set. Compilation instructions are described in the `README.md` file of the submission package.

7.1 Key Sizes

The concrete sizes are listed in Table 7.1 for each security level. We also present the theoretical compressed public key sizes, which is attractive among post-quantum cryptography. In the next version we will propose the compressed version of NIIKE and the concrete compressed public key sizes.

Table 7.1: Theoretical and implementation key sizes of NIIKE targeting all different security levels.

Setting	Private key	Public key		Shared secret key
		Current Impl.	Theo. Compressed	
NGCC-I	196B	4030B	649B	62B
NGCC-II	332B	8943B	1323B	128B
NGCC-III	3032B	40980B	4193B	2049B

7.2 Performance Evaluation

All benchmarks are executed on an Intel Core i7-11700F CPU with a base frequency of 2.50 GHz. To ensure the stability and reproducibility of the test environment, neither hyperthreading nor turbo boost technology is available. The operating system is Ubuntu

24.04 LTS, and the code is compiled with Clang 18.1.3 at the `-O3` optimization level with link-time optimization enabled. The NGCC-I AVX-512IFMA implementation additionally enables the `-march=native` flag.

Table 7.2: NIIKE performance in 10^9 CPU cycles on an Intel Core i7-11700F CPU. Results (expressed in gigacycles) are the median of 100 benchmark runs.

Setting	Key Generation	Key Agreement
<i>Reference implementation</i>		
NGCC-I	18.37	15.14
NGCC-II	428.60	375.70
NGCC-III	68323.43	67945.89
<i>Optimized implementation (x86 assembly)</i>		
NGCC-I	13.49	11.19
NGCC-II	261.98	238.53
NGCC-III	68239.54	67643.43
<i>Additional implementation (AVX-512IFMA)</i>		
NGCC-I	2.82	2.50

CHAPTER 8

Implementation details

8.1 Finite Field Arithmetic

Since isogeny computations account for the majority of the execution time of NIIKE, including both key generation and key agreement, and since these computations rely heavily on arithmetic in $\mathbb{F}_{p^2}$ and its base field $\mathbb{F}_p$, faster arithmetic in these fields directly translates into better NIIKE performance. Unlike CSIDH, which operates over $\mathbb{F}_p$, the NIIKE implementation targeting NGCC-I and NGCC-II security levels requires arithmetic over $\mathbb{F}_{p^2}$ due to its use of supersingular elliptic curves oriented by imaginary quadratic orders of large discriminant. Although this increases the complexity of the base field arithmetic, it permits the use of a much smaller prime p , which provides a substantial performance lever.

Extension field arithmetic is built on top of $\mathbb{F}_p$ using the standard Karatsuba or school-book approach with $i^2 = -1$. Each $\mathbb{F}_{p^2}$ element is stored as a pair of $\mathbb{F}_p$ elements.

8.2 Elliptic Curve Arithmetic

The elliptic curve arithmetic employs the SQIsign library [AAA⁺24]. Accordingly, the description in this section is based on the SQIsign specification.

In our implementation, all elliptic-curve operations are performed on Montgomery curves $y^2 = x^3 + Ax^2 + x$ with $A \in \mathbb{F}_{p^2}$. A curve is represented in projective form by the pair $(A : C)$, where A/C denotes the Montgomery coefficient. We also define $(A_{24} : C_{24}) = (A + 2C : 4C)$.

8.2.1 x -Only Arithmetic

In our implementation, the y -coordinates are omitted because in the class group action procedures they are not necessary for elliptic curve arithmetic. To eliminate inversions,

points are represented in projective form $(X : Z)$. In the following we propose three algorithms: **xDBL** executes point doubling, **xADD** computes the sum of two points where their difference is known, and **xDBLADD** performs one point doubling and addition of two points.

Algorithm 10 $\text{xDBL}^*(P, (A_{24} : C_{24}))$

Input: $P = (X_P : Z_P)$, Montgomery constants $(A_{24} : C_{24})$ of curve E

Output: $[2]P = (X_{2P} : Z_{2P})$

- 1: $t_0 \leftarrow X_P + Z_P$
 - 2: $t_0 \leftarrow t_0^2$
 - 3: $t_1 \leftarrow X_P - Z_P$
 - 4: $t_1 \leftarrow t_1^2$
 - 5: $t_2 \leftarrow t_0 - t_1$
 - 6: $t_1 \leftarrow t_1 \cdot C_{24}$
 - 7: $X_{2P} \leftarrow t_0 \cdot t_1$
 - 8: $t_0 \leftarrow t_2 \cdot A_{24}$
 - 9: $t_0 \leftarrow t_0 + t_1$
 - 10: $Z_{2P} \leftarrow t_0 \cdot t_2$
 - 11: **Return** $(X_{2P} : Z_{2P})$
-

Algorithm 11 $\text{xADD}^*(P, Q, P - Q)$

Input: Projective points $P = (X_P : Z_P)$, $Q = (X_Q : Z_Q)$, $P - Q = (X_{P-Q} : Z_{P-Q})$

Output: $P + Q = (X_{P+Q} : Z_{P+Q})$

- 1: $t_0 \leftarrow X_P + Z_P$
 - 2: $t_1 \leftarrow X_P - Z_P$
 - 3: $t_2 \leftarrow X_Q + Z_Q$
 - 4: $t_3 \leftarrow X_Q - Z_Q$
 - 5: $t_0 \leftarrow t_0 \cdot t_3$
 - 6: $t_1 \leftarrow t_1 \cdot t_2$
 - 7: $t_2 \leftarrow t_0 + t_1$
 - 8: $t_3 \leftarrow t_0 - t_1$
 - 9: $t_2 \leftarrow t_2^2$
 - 10: $t_3 \leftarrow t_3^2$
 - 11: $X_{P+Q} \leftarrow Z_{P-Q} \cdot t_2$
 - 12: $Z_{P+Q} \leftarrow X_{P-Q} \cdot t_3$
 - 13: **Return** $P + Q = (X_{P+Q} : Z_{P+Q})$
-

Algorithm 12 xDBLADD*($P, Q, P - Q, (A_{24} : C_{24})$)

Input: $P = (X_P : Z_P)$, $Q = (X_Q : Z_Q)$, $P - Q = (X_{P-Q} : Z_{P-Q})$, Montgomery constants $(A_{24} : C_{24})$ of E

Output: $[2]P = (X_{2P} : Z_{2P})$ and $P + Q = (X_{P+Q} : Z_{P+Q})$

- 1: $t_0 \leftarrow X_P + Z_P$
 - 2: $t_1 \leftarrow X_P - Z_P$
 - 3: $X_{2P} \leftarrow t_0^2$
 - 4: $t_2 \leftarrow X_Q - Z_Q$
 - 5: $X_{P+Q} \leftarrow X_Q + Z_Q$
 - 6: $t_0 \leftarrow t_0 \cdot t_2$
 - 7: $Z_{2P} \leftarrow t_1^2$
 - 8: $t_1 \leftarrow t_1 \cdot X_{P+Q}$
 - 9: $t_2 \leftarrow X_{2P} - Z_{2P}$
 - 10: $Z_{2P} \leftarrow Z_{2P} \cdot C_{24}$
 - 11: $X_{2P} \leftarrow X_{2P} \cdot Z_{2P}$
 - 12: $X_{P+Q} \leftarrow A_{24} \cdot t_2$
 - 13: $Z_{P+Q} \leftarrow t_0 - t_1$
 - 14: $Z_{2P} \leftarrow Z_{2P} + X_{P+Q}$
 - 15: $X_{P+Q} \leftarrow t_0 + t_1$
 - 16: $Z_{2P} \leftarrow Z_{2P} \cdot t_2$
 - 17: $Z_{P+Q} \leftarrow Z_{P+Q}^2$
 - 18: $X_{P+Q} \leftarrow X_{P+Q}^2$
 - 19: $Z_{P+Q} \leftarrow Z_{P+Q} \cdot X_{P-Q}$
 - 20: $X_{P+Q} \leftarrow X_{P+Q} \cdot Z_{P-Q}$
 - 21: **Return** $((X_{2P} : Z_{2P}), (X_{P+Q} : Z_{P+Q}))$
-

LADDER computes $[m]P$ given a positive scalar m and a point P . More precisely, taken a point $P = (X_P : Z_P)$, the coefficient $(A_{24} : C_{24})$ of the curve, and a positive scalar m as input, and outputs the projective x -coordinate of $[m]P$.

Algorithm 13 LADDER^{*}(P, E, m)

Input: A projective point $P = (X_P : Z_P)$, the coefficient $(A_{24} : C_{24})$ of the curve E , and a positive scalar m with binary representation $m = (m_{k-1}, \dots, m_0)_2$

Output: $[m]P = (X_{[m]P} : Z_{[m]P})$

```

1:  $((X_0 : Z_0), (X_1 : Z_1)) \leftarrow ((1 : 0), (X_P : Z_P))$ 
2: For  $i$  from  $k - 1$  down to 0 do
3:   If  $m_i = 1$  then
4:      $((X_1, Z_1), (X_0, Z_0)) \leftarrow \text{xDBLADD}((X_1 : Z_1), (X_0 : Z_0), (X_P : Z_P), (A_{24} : C_{24}))$ 
5:   else
6:      $((X_0, Z_0), (X_1 : Z_1)) \leftarrow \text{xDBLADD}((X_0 : Z_0), (X_1 : Z_1), (X_P : Z_P), (A_{24} : C_{24}))$ 
7:   end If
8: end For
9:  $(X_{[m]P} : Z_{[m]P}) \leftarrow (X_0 : Z_0)$ 
10: Return  $[m]P = (X_{[m]P} : Z_{[m]P})$ 

```

8.3 Isogenies in Dimension One

NIIKE requires smooth odd-degree isogenies in dimension one for evaluating endomorphisms. For each prime $\ell = 2d + 1$, the isogeny is computed using a hybrid Montgomery–Twisted Edwards framework [CH17]. Specifically, Montgomery projective coordinates $(X : Z)$ are transformed into Twisted Edwards coordinates through the mapping $y = (X - Z, X + Z)$. Within the Edwards model, the kernel point set is efficiently constructed by **KERNEL-POINTSET**, which is based on **yDBL** and **yADD**.

Algorithm 14 $\text{yDBL}^*(P, (A : C))$ **Input:** Twisted Edwards point $P = (Y_P : Z_P)$, Montgomery constants $(A : C)$ of curve E **Output:** $[2]P = (Y_{2P} : Z_{2P})$ in Twisted Edwards form

- 1: $t_0 \leftarrow Y_P^2$
- 2: $t_1 \leftarrow Z_P^2$
- 3: $Z \leftarrow C \cdot t_0$
- 4: $X \leftarrow Z \cdot t_1$
- 5: $t_1 \leftarrow t_1 - t_0$
- 6: $t_0 \leftarrow A \cdot t_1$
- 7: $Z \leftarrow Z + t_0$
- 8: $Z \leftarrow Z \cdot t_1$
- 9: $Y_{2P} \leftarrow X - Z$
- 10: $Z_{2P} \leftarrow X + Z$
- 11: **Return** $(Y_{2P} : Z_{2P})$

Algorithm 15 $\text{yADD}^*(P, Q, P-Q)$ **Input:** Twisted Edwards points $P = (Y_P : Z_P)$, $Q = (Y_Q : Z_Q)$, $P - Q = (Y_{PQ} : Z_{PQ})$ **Output:** $P + Q = (Y_R : Z_R)$ in Twisted Edwards form

- 1: $a \leftarrow Z_P \cdot Y_Q$
- 2: $b \leftarrow Y_P \cdot Z_Q$
- 3: $c \leftarrow (a + b)^2$
- 4: $d \leftarrow (a - b)^2$
- 5: $X \leftarrow (Z_{PQ} - Y_{PQ}) \cdot c$
- 6: $Z \leftarrow (Z_{PQ} + Y_{PQ}) \cdot d$
- 7: $Y_R \leftarrow X - Z$
- 8: $Z_R \leftarrow X + Z$
- 9: **Return** $(Y_R : Z_R)$

Algorithm 16 $\text{KERNELPOINTSET}^*(\ell, P, (A : C))$ **Input:** Odd prime $\ell = 2d + 1$, kernel generator $P = (X_P : Z_P)$ in Montgomery form, curve constants $(A : C)$ **Output:** Set $\{K_j\}_{j=0}^{d-1}$ in Twisted Edwards form

- 1: $K_0 \leftarrow (X_P - Z_P, X_P + Z_P)$
- 2: $K_1 \leftarrow \text{yDBL}(K_0, (A : C))$
- 3: **For** $j = 2, \dots, d - 1$ **do**
- 4: $K_j \leftarrow \text{yADD}(K_{j-1}, K_0, K_{j-2})$
- 5: **end For**
- 6: **Return** $\{K_0, K_1, \dots, K_{d-1}\}$

After computing the kernel point set, one can compute the codomain of the isogeny by [OddIsogenyCodomain](#). Note that the algorithm first computes the codomain in Edwards form, and maps back to Montgomery form. [OddIsogenyEval](#) performs point evaluation under the isogeny by leveraging the `CRISSCROSS` subroutine, which is introduced by Costello and Hisil [CH17].

Algorithm 17 `ODDISOGENYCODOMAIN*`($\ell, (A : C)$)

Input: Odd prime $\ell = 2d + 1$, Montgomery constants $(A : C)$, set $\{K_j\}_{j=0}^{d-1}$

Output: Codomain Montgomery coefficient $(A' : C')$

```

1:  $B_y \leftarrow K_0 \cdot y, \quad B_z \leftarrow K_0 \cdot z$ 
2: For  $j = 1, \dots, d - 1$  do
3:    $B_y \leftarrow B_y \cdot K_j \cdot y, \quad B_z \leftarrow B_z \cdot K_j \cdot z$ 
4: end For
5:  $d_E \leftarrow A - C$ 
6:  $a_E \leftarrow A, \quad d'_E \leftarrow d_E$ 
7: For  $j = 1, \dots, \lceil \log_2 \ell \rceil - 1$  do
8:    $a_E \leftarrow a_E^2, \quad d'_E \leftarrow d'^2_E$ 
9:   If bit  $j$  of  $\ell$  is 1 then
10:     $a_E \leftarrow a_E \cdot A, \quad d'_E \leftarrow d'_E \cdot d_E$ 
11:   end If
12: end For
13:  $B_y \leftarrow B_y^8, \quad B_z \leftarrow B_z^8$ 
14:  $A' \leftarrow a_E \cdot B_z$ 
15:  $C' \leftarrow A' - d'_E \cdot B_y$ 
16: Return  $(A' : C')$ 

```

Algorithm 18 `CRISSCROSS*`($\alpha, \beta, \gamma, \delta$)

Input: $\alpha, \beta, \gamma, \delta \in \mathbb{F}_{p^2}$

Output: (r_0, r_1) with $r_0 = \alpha\delta + \beta\gamma, r_1 = \alpha\delta - \beta\gamma$

```

1:  $t_1 \leftarrow \alpha \cdot \delta$ 
2:  $t_2 \leftarrow \beta \cdot \gamma$ 
3:  $r_0 \leftarrow t_1 + t_2$ 
4:  $r_1 \leftarrow t_1 - t_2$ 
5: Return  $(r_0, r_1)$ 

```

Algorithm 19 $\text{ODDISOGENYEVAL}^*(\ell, P)$

Input: Odd prime $\ell = 2d + 1$, projective point $P = (X_P : Z_P)$, set $\{K_j\}_{j=0}^{d-1}$
Output: $Q = (X_Q : Z_Q) \in E'$ of P under the ℓ -isogeny

- 1: $S_0 \leftarrow X_P + Z_P, S_1 \leftarrow X_P - Z_P$
- 2: $(R_0, R_1) \leftarrow \text{CRISSCROSS}(K_0.z, K_0.y, S_0, S_1)$
- 3: **For** $j = 1, \dots, d - 1$ **do**
- 4: $(T_0, T_1) \leftarrow \text{CRISSCROSS}(K_j.z, K_j.y, S_0, S_1)$
- 5: $R_0 \leftarrow R_0 \cdot T_0, R_1 \leftarrow R_1 \cdot T_1$
- 6: **end For**
- 7: $R_0 \leftarrow R_0^2, R_1 \leftarrow R_1^2$
- 8: $X_Q \leftarrow X_P \cdot R_0, Z_Q \leftarrow Z_P \cdot R_1$
- 9: **Return** $(X_Q : Z_Q)$

Algorithm 20 $\text{ISOGENY}(E, S, K, \ell)$

Input: curve constants $(A : C)$, projective points $S = \{P_1, P_2, \dots, P_n\}$, kernel generator K , odd prime ℓ
Output: Codomain Montgomery coefficient $E' = (A' : C')$, points $\{Q_1, Q_2, \dots, Q_n\} \in E'$ of $\{P_1, P_2, \dots, P_n\}$ under the ℓ -isogeny with kernel $\langle K \rangle$

- 1: $\{K_j\}_{j=0}^{d-1} \leftarrow \text{KERNELPOINTSET}(\ell, P, (A : C))$
- 2: $E' \leftarrow \text{ODDISOGENYCODOMAIN}(\ell, (A : C))$
- 3: **For** $i = 1, 2, \dots, n$ **do**
- 4: $Q_i \leftarrow \text{ODDISOGENYEVAL}(\ell, P_i)$
- 5: **end For**
- 6: **Return** $E', \{Q_1, Q_2, \dots, Q_n\}$

In addition to odd-degree isogenies, the NGCC-III implementation computes an extra isogeny of degree 4 with kernel $\langle (1, _) \rangle$ to ensure the dummy-free property of NIIKE. For any point P defined on the curve $E_A : y^2 = x^3 + Ax^2 + x$, EVALFOURISOG evaluates this 4-isogeny at P .

Algorithm 21 EVALFOURISOG(A, P)**Input:** Affine Montgomery curve coefficient A , projective point $P = (X : Z)$ **Output:** The evaluated point $P' = (X' : Z')$ under the 4-isogeny with kernel $\langle(1, _)\rangle$

```

1:  $t_0 \leftarrow X \cdot Z$ 
2:  $X \leftarrow X - Z$ 
3:  $X \leftarrow X^2$ 
4:  $Z \leftarrow t_0$ 
5:  $t_2 \leftarrow A - 2$ 
6:  $t_2 \leftarrow t_2 \cdot X$ 
7:  $t_2 \leftarrow t_2 \cdot Z$ 
8:  $t_1 \leftarrow A + 2$ 
9:  $t_0 \leftarrow Z + Z$ 
10:  $t_0 \leftarrow t_0 + t_0$ 
11:  $t_0 \leftarrow X + t_0$ 
12:  $t_1 \leftarrow t_1 \cdot Z$ 
13:  $t_1 \leftarrow X + t_1$ 
14:  $X' \leftarrow t_0 \cdot t_1$ 
15:  $Z' \leftarrow t_2$ 
16: Return  $(X' : Z')$ 

```

8.4 Parallel Optimizations

The class group action used in NIIKE has a natural task-level parallel structure. Although the complete key generation or key agreement procedure consists of several consecutive applications of the group action, these applications cannot in general be evaluated independently, since each application requires the full kernel representation produced by the preceding one. Therefore, the main source of low-latency parallelism is not obtained by executing several independent protocol instances in parallel, but by decomposing a single group action into independent tuple updates.

Recall that a full kernel representation is written as $\mathcal{E} = \{(E_j, P_j, Q_j) \mid 1 \leq j \leq r\}$. For a fixed secret vector, the computation of the output tuple (E'_j, P'_j, Q'_j) only depends on the two neighboring input tuples (E_j, P_j, Q_j) and $(E_{j+1}, P_{j+1}, Q_{j+1})$, where the indices are interpreted cyclically. In particular, once these two input tuples are available, the corresponding isogeny chains ϕ_j^+ and ϕ_j^- can be computed without accessing the remaining tuples of the representation. Hence, the r tuple updates inside one group action form a collection of independent tasks with the same control flow and comparable arithmetic workload. This observation is the basis of our parallel implementation.

We use a fork-join execution model. At the beginning of one parallel stage, the implementation assigns a set of ready tuple-update tasks to the available workers. Each worker performs the scalar multiplications, isogeny constructions, isogeny evaluations, and the

required codomain identification for its assigned tuple. After all tasks in the stage have completed, the resulting tuples are collected to form the next full kernel representation. This design is particularly suitable for NIIKE because the underlying group action is deterministic and constant-time at the algorithmic level: all tuple updates follow the same sequence of loop iterations, conditional swaps, scalar multiplications, and one-dimensional isogeny computations. Consequently, the scheduling granularity is coarse enough to amortize thread-management overhead, while the arithmetic kernels remain structured enough for vectorized implementations.

Let n_{thr} denote the number of available worker threads. If $n_{\text{thr}} \geq r$, one group action can be evaluated with essentially one tuple per worker. If $n_{\text{thr}} < r$, the r tuple updates are processed in several batches: at most n_{thr} independent updates are launched in each batch, and the next batch starts after the previous batch has joined. For key generation, all r tuples are needed after every group action application, so the amount of ready work remains stable throughout the computation. The expected latency therefore scales approximately with $\lceil r/n_{\text{thr}} \rceil$ parallel batches per group action, up to synchronization overhead, cache contention, and memory-layout effects.

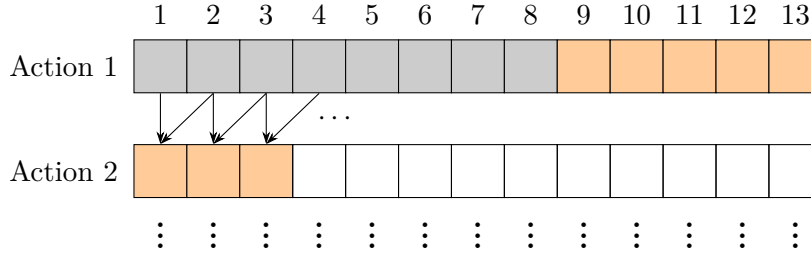


Figure 8.1: Parallel implementation of key generation. We set $n_{\text{thr}} = 8$ and $r = 13$.

The key agreement phase admits an additional optimization due to tail pruning. Since the shared secret is derived only from the final j -invariant, the last several applications of the group action do not require the complete output representation. More precisely, after the pruning step, only a prefix of the tuple sequence must be updated. At the final several applications, the implementation evaluates only the tuple updates that can still influence the final curve used for the shared secret. This reduces the total number of tuple updates by $r(r-1)/2$ over the whole key-agreement computation. The pruning rule depends only on public iteration indices and public parameters, so it preserves the deterministic and dummy-free structure of the protocol.

Tail pruning changes the parallel workload near the end of key agreement. In the early part of the computation, the implementation still has r ready tuple-update tasks and can therefore saturate up to r workers. In the pruned tail, however, the number of necessary updates decreases from one iteration to the next. Once the number of ready updates falls below n_{thr} , some workers remain idle. Thus, key agreement benefits from

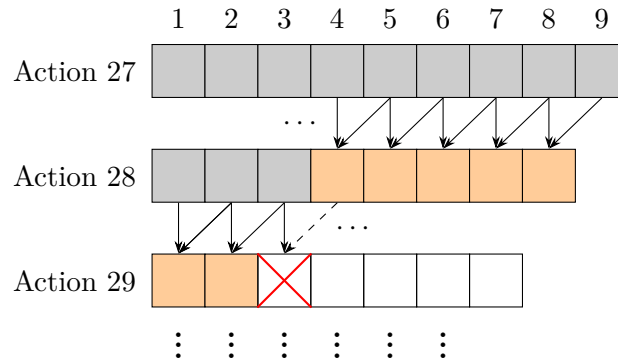


Figure 8.2: Parallel implementation of key agreement. Under the NGCC-I parameter setting, the parameter $B = 35$ and $r = 13$, and we set $n_{\text{thr}} = 8$. The task marked with red crosses cannot be batched in the current execution due to unavailable required tuples.

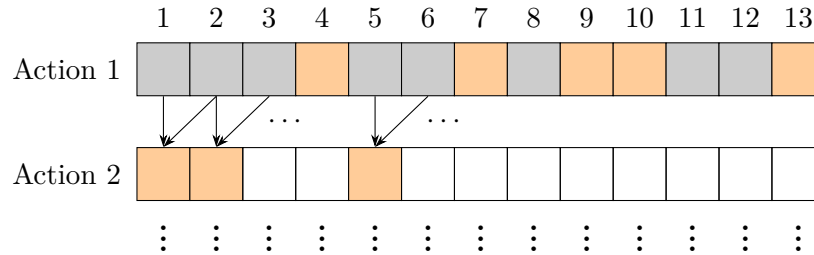


Figure 8.3: An example of thread-scheduling strategy.

both tuple-level parallelism and pruning, but its final iterations no longer exhibit perfect n_{thr} -way parallelism. This explains why the parallel speedup in key agreement is expected to be slightly lower than the ideal speedup obtained in key generation, while still improving the absolute running time.

From an implementation perspective, a static fork-join schedule gives a simple and predictable design. A more dynamic work-queue schedule can reduce idle cycles by allowing a worker to fetch another ready tuple update immediately after finishing its current task. However, such a design introduces additional synchronization, locking, and memory-ordering costs. For the NIIKE parameter sets, the tuple-update tasks are already large and well balanced, so the fork-join model provides a favorable trade-off between parallel efficiency and implementation complexity.

8.4.1 AVX-512 IFMA Implementation

Our parallel strategy supports SIMD acceleration. Since different tuple updates execute the same arithmetic operations on independent data, the implementation can pack several field elements, curve coefficients, or point information into vector lanes and process them simultaneously. This is especially effective for AVX-512IFMA, where one vector instruction can operate on multiple radix- 2^{52} limbs. In this setting, the tuple-level parallelism provides the batching structure required by vector arithmetic, while the one-dimensional isogeny formulas described above supply regular arithmetic kernels for finite-field multiplication, squaring, addition, subtraction, and modular reduction.

Radix- 2^{52} representation. Given that the NIIKE parameter set targeting the NGCC-I security level would result in an extra limb under radix- 2^{51} , the implementation opts for radix- 2^{52} instead. Consequently, an $\mathbb{F}_p$ element f is represented as:

$$f = f_0 + 2^{52}f_1 + 2^{104}f_2 + 2^{156}f_3 + 2^{208}f_4$$

, where $0 \leq f_j < 2^{52}$ for $j = 0, 1, \dots, 4$. This representation allows field elements to be up to 260 bits during intermediate computations (before final reduction).

Limb vector set (8-way parallelism). Given radix- 2^{52} representations of eight $\mathbb{F}_p$ elements a, b, c, d, e, f, g, h , the implementation defines a limb vector set V :

$$V = \langle a, b, c, d, e, f, g, h \rangle = (V_0, V_1, V_2, V_3, V_4),$$

where each $V_i = [a_i, b_i, c_i, d_i, e_i, f_i, g_i, h_i]$ is a limb vector. Performing addition, subtraction, or multiplication on two limb vectors is equivalent to conducting these operations on two integers modulo p .

In short, the AVX-512 IFMA implementation enables eight identical $\mathbb{F}_p$ (or $\mathbb{F}_{p^2}$) operations to execute simultaneously. Table 8.1 reports the theoretical acceleration factor analysis of the AVX512-IFMA implementation, by taking multiplication as the representative operation (also the dominant cost):

Table 8.1: Theoretical speedup analysis (multiplication)

Metric	x64 Assembly	AVX-512 IFMA
Limbs pre $\mathbb{F}_p$ element	4 (64-bit)	5 (52-bit)
Word multiplications per $\mathbb{F}_p$ mul	$(256/64)^2 = 16$	$5^2 = 25$
Parallel lanes	1	8
Theoretical optimal speedup	-	$\frac{16}{25} \times 8 = 5.12$

The experimental results confirm this analysis: for NGCC-I security level, the AVX-512IFMA implementation achieves a $4.78\times$ speedup in key generation (near-perfect parallelism) and a $4.48\times$ speedup in key agreement. The slightly lower gain in key agreement is attributable to the tail pruning technique, which causes the last several iterations to have reduced parallelism.

8.4.2 AVX-512IFMA Support Notes

AVX-512IFMA has not yet been fully adopted in consumer desktop processors:

- AMD: Full 512-bit data-path support for AVX-512 (including IFMA extensions) has been available since the Zen 5 architecture.
- Intel: AVX-512 support on desktop processors was discontinued after the 11th-generation Core series. It remains available only on server- and workstation-grade processors. The upcoming Nova Lake series will reintroduce AVX-512 support through the AVX10.2 extension, which includes IFMA instructions.

CHAPTER 9

Security

9.1 Theoretical Security

This section defines the security assumptions used by NIIKE. The following problem is the basic problem in the group-action setting, and NIIKE is based on an equivalent problem.

9.1.1 Computational Assumptions

Problem 9.1.1 (Group Action Inversion Problem). Let $\mathbf{pp}$ be the public parameters, and let $\mathcal{E}_0$ be the distinguished origin. Given $\mathcal{E}' = s * \mathcal{E}_0$, where s is sampled uniformly at random from $\text{Cl}(\mathcal{O})$, find an element $s' \in \text{Cl}(\mathcal{O})$ such that $\mathcal{E}' = s' * \mathcal{E}_0$.

Problem 9.1.2 (Restricted Group Action Computational Diffie–Hellman Problem). Let $\mathbf{pp}$ be the public parameters, and let $\mathcal{E}_0$ be the distinguished origin. Given $\mathcal{E}_A = a * \mathcal{E}_0$ and $\mathcal{E}_B = b * \mathcal{E}_0$, where a and b are sampled independently and uniformly at random from $[0, Be_i]_{i=1}^n$, find an elliptic curve $\mathcal{E}'$ such that $\mathcal{E}' \cong (ab) * \mathcal{E}_0$.

We say that Problem 9.1.2 is ε -hard if no probabilistic polynomial-time adversary can solve the problem with probability greater than ε .

The two problems are quantumly equivalent. The equivalence is given for a perfect oracle in [GPSV18], proved for an imperfect oracle in [MZ24], and tightened in [GLM24].

Theorem 9.1.3. *For given public parameters $\mathbf{pp}$ with $\mathcal{E}_0$ as the distinguished origin, the existence of a PPT adversary against the Group Action Computational Diffie–Hellman problem implies the existence of a PPT adversary against the Group Action Inversion problem.*

9.1.2 Security Notions and Security Proof

This subsection defines the security notion for the scheme for non-interactive key exchange used in [CKS08].

Definition 9.1.4 (Non-Interactive Key Exchange). A non-interactive key exchange scheme is a tuple

$$\Pi_{\text{NIKE}} := (\text{Stp}, \text{KeyGen}, \text{KeyAgr})$$

of probabilistic polynomial-time algorithms, together with an identity space $\mathcal{ID}$ and a shared-key space $\mathcal{SK}$, defined as follows:

- $\text{pp} \leftarrow \text{Stp}(1^\lambda)$: On input the security parameter 1^λ , the setup algorithm outputs public parameters pp .
- $(\text{sk}, \text{pk}) \leftarrow \text{KeyGen}(\text{ID})$: On input an identity $\text{ID} \in \mathcal{ID}$, the key-generation algorithm outputs a secret/public key pair (sk, pk) .
- $k \leftarrow \text{KeyAgr}(\text{ID}_1, \text{pk}_1, \text{ID}_2, \text{sk}_2)$: On input an identity $\text{ID}_1 \in \mathcal{ID}$ together with its corresponding public key pk_1 , and another identity $\text{ID}_2 \in \mathcal{ID}$ together with its corresponding secret key sk_2 , the shared-key derivation algorithm outputs either a shared key $k \in \mathcal{SK}$ or a failure symbol $\perp$.

We assume that KeyAgr always outputs $\perp$ if $\text{ID}_1 = \text{ID}_2$.

Definition 9.1.5 (Correctness). A scheme $\Pi_{\text{NIKE}} := (\text{Stp}, \text{KeyGen}, \text{KeyAgr})$ has honest correctness error δ , or is said to be perfectly correct, if for all $\text{pp} \in \text{Stp}(1^\lambda)$ and all $\text{ID}_1, \text{ID}_2 \in \mathcal{ID}$,

$$\Pr \left[\text{KeyAgr}(\text{ID}_1, \text{pk}_1, \text{ID}_2, \text{sk}_2) \neq \text{KeyAgr}(\text{ID}_2, \text{pk}_2, \text{ID}_1, \text{sk}_1) \mid \begin{array}{l} (\text{sk}_1, \text{pk}_1) \leftarrow \text{KeyGen}(\text{ID}_1), \\ (\text{sk}_2, \text{pk}_2) \leftarrow \text{KeyGen}(\text{ID}_2) \end{array} \right] = 0,$$

where the probability is taken over the random coins of Stp and KeyGen .

Definition 9.1.6 (Passive Security (HKR-CKS-1)). Let $\Pi_{\text{NIKE}} := (\text{Stp}, \text{KeyGen}, \text{KeyAgr})$ be a non-interactive key exchange scheme. We say that Π_{NIKE} is (ε, Q_H) -HKR-CKS-I secure relative to $\text{pp} \in \text{Stp}(1^\lambda)$ if for all probabilistic polynomial-time adversaries $\mathcal{A}$ making at most

- Q_H queries to the random oracle H ,
- two queries to the RegHonUsr oracle, and
- one query to the TestQue oracle,

we have

$$\text{Adv}_{\Pi_{\text{NIKE}}, \text{pp}}^{\text{HKR-CKS-I}}(\mathcal{A}) \leq \varepsilon(Q_H),$$

where the experiment $\text{HKR-CKS-I}_{\Pi_{\text{NIKE}}, \text{pp}}^{\mathcal{A}}$ is defined as follows. The challenger samples a hidden bit $b \xleftarrow{\$} \{0, 1\}$. The adversary may query the RegHonUsr oracle on identities $\text{ID} \in \mathcal{ID}$. In response, the challenger runs $(\text{sk}, \text{pk}) \xleftarrow{\$} \text{KeyGen}(\text{ID})$ and returns pk to the adversary while storing $(\text{ID}, \text{pk}, \text{sk})$. After two honest registrations, the adversary may issue

one **TestQue** query on two distinct registered identities ID_1, ID_2 . In response, the challenger returns

$$K_b := \begin{cases} \text{KeyAgr}(ID_1, pk_1, ID_2, sk_2) & \text{if } b = 0, \\ K \xleftarrow{\$} \mathcal{SK} & \text{if } b = 1, \end{cases}$$

where (ID_1, pk_1, sk_1) and (ID_2, pk_2, sk_2) are the stored key pairs corresponding to ID_1 and ID_2 . Finally, the adversary outputs a bit b' , and the experiment outputs 1 if and only if $b' = b$. The advantage of $\mathcal{A}$ is defined by

$$\text{Adv}_{\Pi_{\text{NIKE}}, \text{pp}}^{\text{HKR-CKS-I}}(\mathcal{A}) := \left| \Pr \left[\text{HKR-CKS-I}_{\Pi_{\text{NIKE}}, \text{pp}}^{\mathcal{A}} \Rightarrow 1 \right] - \frac{1}{2} \right|.$$

The proof is given directly in the random-oracle model.

Theorem 9.1.7. *If Problem 9.1.2 is ε -hard relative to pp , then for every probabilistic polynomial-time adversary $\mathcal{A}$ making at most Q_H queries to H , there exists a probabilistic polynomial-time adversary $\mathcal{B}$ such that $\text{Adv}_{\Pi_{\text{NIKE}}, \text{pp}}^{\text{HKR-CKS-I}}(\mathcal{A}) \leq \varepsilon Q_H$. In particular, Π_{NIKE} is HKR-CKS-I secure relative to pp .*

Proof. Let $\mathcal{A}$ be a PPT adversary against HKR-CKS-I making at most Q_H queries to the random oracle H . We construct a PPT adversary $\mathcal{B}$ for Problem 9.1.2.

Adversary $\mathcal{B}$ embeds its challenge instance into the two honestly generated public keys returned to $\mathcal{A}$ through the **RegHonUsr** oracle. When $\mathcal{A}$ issues its **TestQue** query on the challenge pair, the experiment returns either the real session key digest $H(K)$ or a uniformly random element of the output space of H , where $K := \text{KeyAgr}(ID_1, pk_1, ID_2, sk_2)$.

Adversary $\mathcal{B}$ answers the random-oracle queries of $\mathcal{A}$ by lazy sampling. As long as $\mathcal{A}$ does not query H on the correct shared key K , the value returned by **TestQue** is independent of the hidden bit from the viewpoint of $\mathcal{A}$. Therefore, any non-negligible advantage of $\mathcal{A}$ in distinguishing the two branches implies that $\mathcal{A}$ queries the random oracle on the correct shared key with the same probability.

Adversary $\mathcal{B}$ records all random-oracle queries made by $\mathcal{A}$, guesses one of them uniformly at random, and outputs it as its candidate solution to Problem 9.1.2. If $\mathcal{A}$ queries H on the correct shared key, then $\mathcal{B}$ succeeds with probability at least $1/Q_H$. It follows that $\text{Adv}_{\Pi_{\text{NIKE}}, \text{pp}}^{\text{HKR-CKS-I}}(\mathcal{A})$ is bounded by Q_H times the advantage of $\mathcal{B}$ against Problem 9.1.2.

Hence the assumed hardness of Problem 9.1.2 implies the claimed HKR-CKS-I security of Π_{NIKE} . $\square$

9.2 Practical Security

The practical security of **NIKE** relies on the following parameters:

- The class number $\#\text{Cl}(\mathcal{O})$;
- The public key space size $\#\mathcal{K}_{\text{pk}}$;

- The characteristic p .

Note that the class number $\#\text{Cl}(\mathcal{O})$ is roughly proportional to $|\text{Disc}(\mathcal{O})|^{1/2}$. Same as CSIDH, we need to confirm that the orientation is large enough to resist Kuperberg’s algorithm [Kup05]. According to the conservative analysis in [BLMP19, Pei20, BS20], to target NGCC-I security level $|\text{Disc}(\mathcal{O})| \approx 2^{4096}$. Heuristically, we expect that $|\text{Disc}(\mathcal{O})| \approx 2^{8192}$ and $|\text{Disc}(\mathcal{O})| \approx 2^{16384}$ to target NGCC-II and NGCC-III security levels, respectively.

Recall from Equation (4.1) that the private key space is defined as

$$\mathcal{K}_{\text{priv}} = \{(s_1, \dots, s_n) \in \mathbb{Z}^n \mid 0 \leq s_i \leq Be_i\}.$$

with size larger than $2^{2\lambda}$. Each private key corresponds to an ideal class $[\prod_{i=1}^n \mathfrak{l}_i^{s_i}] \in \text{Cl}(\mathcal{O})$ [CSCDJRH22]. As with CSIDH, we rely on the heuristic assumption that this correspondence is nearly injective, meaning that collisions in the mapping from the private key space to the class group are rare. This assumption is supported by the Cohen–Lenstra heuristics [CL84] combined with the Gaussian heuristic applied to the relation lattice spanned by the ideal classes $[\mathfrak{l}_i]$. Letting $\mathcal{S} \subseteq \text{Cl}(\mathcal{O})$ denote the image of $\mathcal{K}_{\text{priv}}$ under this map, the public key space is defined as

$$\mathcal{K}_{\text{pub}} = \{g * \mathcal{E}_0 \mid g \in \mathcal{S}\}.$$

The freeness of the class group action ensures that the mapping $g \mapsto g * \mathcal{E}_0$ is injective, but this does not necessarily extend to the underlying j -invariants. In fact, if $\#\text{Cl}(\mathcal{O}) > p$, the orbit of a **primitively** $\mathcal{O}$ -oriented supersingular curve under the class group action will inevitably include distinct oriented curves that share the same j -invariant but possess different orientations. Consequently, for relatively small p , the security analysis must incorporate generic supersingular path-finding algorithms. Indeed, the problem of finding an ideal class connecting two $\mathcal{O}$ -oriented curves is equivalent to the broader task of finding an arbitrary (not necessarily oriented) isogeny between their underlying curves [Wes22]. Therefore, we need to consider the generic isogeny attacks [BJS14], limiting that $p \gtrsim 2^{2\lambda}$.

Table 9.1: Parameter sizes targeting different NGCC security levels.

Security Level	$ \text{Disc}(\mathcal{O}) $	$\#\mathcal{K}_{\text{pk}}$	$\log_2(p)$
NGCC-I	$\gtrsim 2^{4096}$	$\approx 2^{256}$	$\gtrsim 2^{256}$
NGCC-II	$\gtrsim 2^{8192}$	$\approx 2^{512}$	$\gtrsim 2^{512}$
NGCC-III	$\gtrsim 2^{16384}$	$\approx 2^{1024}$	$\gtrsim 2^{1024}$

As detailed in Chapter 5, our proposed parameter sets successfully satisfy all targeted security levels.

9.3 Other Security Properties

The ideal class group action of NIIKE is deterministic, constant-time and dummy-free. Therefore, the implementation is inherently resistant to a wide class of side-channel attacks (SCA). By ensuring that our subroutines are constant-time, deterministic, and dummy-free, we achieve execution-level robustness without the overhead of traditional masking or blinding countermeasures.

- **Timing attacks:** The NIIKE protocol is implemented in constant-time, meaning the execution flow and instruction sequence are independent of the secret input. By avoiding conditional branches and secret-dependent loop bounds, we mitigate timing attacks. In the context of isogeny-based cryptography, this ensures that an adversary cannot deduce the secret isogeny path or the structure of the kernel by measuring the time required for point evaluation or coordinate transformations.
- **Simple power analysis (SPA):** In a dummy-free architecture, every executed instruction contributes to the final result. This property is critical for thwarting simple power analysis. Since there are no "fake" operations, no observable power consumption patterns or "gaps" that would allow an attacker to distinguish between different secret-dependent execution paths.
- **Fault injection attacks:** The combination of dummy-free and deterministic properties provides a robust defense against Fault injection attack (FIA). In implementations that use dummy operations, an attacker can inject a fault into a specific instruction. If the output remains correct, the attacker learns that the instruction was a dummy, thereby leaking information about the secret branch. Because every operation in NIIKE is essential to the computation (dummy-free), any injected fault will necessarily corrupt the final output, providing no information leakage to the adversary.

CHAPTER 10

Failure Analysis

Unlike certain key exchange protocols that inherently suffer from a non-zero failure rate, NIIKE guarantees flawless execution. As long as both parties correctly sample their random secret keys, they will flawlessly derive the identical shared secret without exception. This zero failure rate significantly reinforces the overall security of the scheme, as it completely eliminates attack vectors associated with failure-boosting or statistical analysis on protocol errors.

Bibliography

- [AAA⁺24] Marius A. Aardal, Gora Adj, Diego F. Aranha, Andrea Basso, Isaac Andrés Canales Martínez, Jorge Chávez-Saab, Maria Corte-Real Santos, Pierrick Dartois, Luca De Feo, Max Duparc, Jonathan Komada Eriksen, Tako Boris Fouotsa, Décio Luiz Gazzoni Filho, Basil Hess, David Kohel, Antonin Leroux, Patrick Longa, Luciano Maino, Michael Meyer, Kohei Nakagawa, Hiroshi Onuki, Lorenz Panny, Sikhar Patranabis, Christophe Petit, Giacomo Pope, Krijn Reijnders, Damien Robert, Francisco Rodríguez Henríquez, Sina Schaeffler, and Benjamin Wesolowski. SQIsign. Technical report, National Institute of Standards and Technology, 2024. available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures>.
- [BBC⁺21] Gustavo Banegas, Daniel J. Bernstein, Fabio Campos, Tung Chou, Tanja Lange, Michael Meyer, Benjamin Smith, and Jana Sotáková. CTIDH: faster constant-time CSIDH. *IACR TCHES*, 2021(4):351–387, 2021.
- [BDFLS20] Daniel J. Bernstein, Luca De Feo, Antonin Leroux, and Benjamin Smith. Faster computation of isogenies of large prime degree. In Steven Galbraith, editor, *ANTS-XIV - 14th Algorithmic Number Theory Symposium*, volume 4 of *Proceedings of the Fourteenth Algorithmic Number Theory Symposium (ANTS-XIV)*, pages 39–55, Auckland, New Zealand, June 2020. Mathematical Sciences Publishers.
- [BHS25] Gustavo Banegas, Andreas Hellenbrand, and Matheus Saldanha. Hardened CTIDH: Dummy-free and deterministic CTIDH. In Ratna Dutta, Luca De Feo, and Sugata Gangopadhyay, editors, *INDOCRYPT 2025*, volume 16372 of *LNCS*, pages 194–215, Bhubaneshwar, India, December 14–17, 2025. Springer, Cham, Switzerland.
- [BJS14] Jean-François Biasse, David Jao, and Anirudh Sankar. A Quantum Algorithm for Computing Isogenies between Supersingular Elliptic Curves. In Willi Meier and Debdeep Mukhopadhyay, editors, *Progress in Cryptology*

- *INDOCRYPT 2014*, pages 428–442, Cham, 2014. Springer International Publishing.
- [BLMP19] Daniel J. Bernstein, Tanja Lange, Chloe Martindale, and Lorenz Panny. Quantum Circuits for the CSIDH: Optimizing Quantum Evaluation of Isogenies. In Yuval Ishai and Vincent Rijmen, editors, *EUROCRYPT 2019, Part II*, volume 11477 of *LNCS*, pages 409–441, Darmstadt, Germany, May 19–23, 2019. Springer, Cham, Switzerland.
- [BS20] Xavier Bonnetain and André Schrottenloher. Quantum Security Analysis of CSIDH. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part II*, volume 12106 of *LNCS*, pages 493–522, Zagreb, Croatia, May 10–14, 2020. Springer, Cham, Switzerland.
- [CCSC⁺24] Fabio Campos, Jorge Chávez-Saab, Jesús-Javier Chi-Domínguez, Michael Meyer, Krijn Reijnders, Francisco Rodríguez-Henríquez, Peter Schwabe, and Thom Wiggers. Optimizations and practicality of high-security CSIDH. *CiC*, 1(1):5, 2024.
- [CH17] Craig Costello and Hüseyin Hisil. A simple and compact algorithm for SIDH with arbitrary degree isogenies. In Tsuyoshi Takagi and Thomas Peyrin, editors, *ASIACRYPT 2017, Part II*, volume 10625 of *LNCS*, pages 303–329, Hong Kong, China, December 3–7, 2017. Springer, Cham, Switzerland.
- [CHMR25] Fabio Campos, Andreas Hellenbrand, Michael Meyer, and Krijn Reijnders. dCTIDH: Fast & deterministic CTIDH. *IACR TCHES*, 2025(3):516–541, 2025.
- [CKS08] David Cash, Eike Kiltz, and Victor Shoup. The twin Diffie-Hellman problem and applications. In Nigel P. Smart, editor, *EUROCRYPT 2008*, volume 4965 of *LNCS*, pages 127–145, Istanbul, Turkey, April 13–17, 2008. Springer Berlin Heidelberg, Germany.
- [CL84] H. Cohen and H. W. Lenstra. Heuristics on class groups of number fields. In Hendrik Jager, editor, *Number Theory Noordwijkerhout 1983*, pages 33–62, Berlin, Heidelberg, 1984. Springer Berlin Heidelberg.
- [CLM⁺18] Wouter Castryck, Tanja Lange, Chloe Martindale, Lorenz Panny, and Joost Renes. CSIDH: An efficient post-quantum commutative group action. In Thomas Peyrin and Steven Galbraith, editors, *ASIACRYPT 2018, Part III*, volume 11274 of *LNCS*, pages 395–427, Brisbane, Queensland, Australia, December 2–6, 2018. Springer, Cham, Switzerland.

- [Cor08] Giuseppe Cornacchia. Su di un metodo per la risoluzione in numeri interi dell'equazione $\sum_{h=0}^n c_h x^{n-h} y^h = p$. *Giornale di Matematiche di Battaglini*, 46:33–90, 1908.
- [CS18] Craig Costello and Benjamin Smith. Montgomery curves and their arithmetic. *Journal of Cryptographic Engineering*, 8(3):227–240, Sep 2018.
- [CSCDJRH22] Jorge Chávez-Saab, Jesús-Javier Chi-Domínguez, Samuel Jaques, and Francisco Rodríguez-Henríquez. The sqale of csidh: sublinear vélu quantum-resistant isogeny action with low exponents. *Journal of Cryptographic Engineering*, 12(3):349–368, Sep 2022.
- [GdKQ⁺24] Phillip Gajland, Bor de Kock, Miguel Quaresma, Giulio Malavolta, and Peter Schwabe. SWOOSH: Efficient lattice-based non-interactive key exchange. In Davide Balzarotti and Wenyuan Xu, editors, *USENIX Security 2024*, Philadelphia, PA, USA, August 14–16, 2024. USENIX Association.
- [GLM24] Steven D. Galbraith, Yi-Fu Lai, and Hart Montgomery. A simpler and more efficient reduction of DLog to CDH for abelian group actions. In Qiang Tang and Vanessa Teague, editors, *PKC 2024, Part II*, volume 14603 of *LNCS*, pages 36–60, Sydney, NSW, Australia, April 15–17, 2024. Springer, Cham, Switzerland.
- [GPSV18] Steven Galbraith, Lorenz Panny, Benjamin Smith, and Frederik Vercauteren. Quantum equivalence of the DLP and CDHP for group actions. *Cryptology ePrint Archive*, Report 2018/1199, 2018.
- [Hou25a] Marc Houben. Deterministic algorithms for class group actions. In Yael Tsauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part I*, volume 16000 of *LNCS*, pages 100–130, Santa Barbara, CA, USA, August 17–21, 2025. Springer, Cham, Switzerland.
- [Hou25b] Marc Houben. Efficient post-quantum commutative group actions from orientations of large discriminant. In Goichiro Hanaoka and Bo-Yin Yang, editors, *ASIACRYPT 2025, Part IV*, volume 16248 of *LNCS*, pages 141–173, Melbourne, VIC, Australia, December 8–12, 2025. Springer, Singapore, Singapore.
- [Kup05] Greg Kuperberg. A Subexponential-Time Quantum Algorithm for the Dihedral Hidden Subgroup Problem. *SIAM Journal on Computing*, 35(1):170–188, 2005.
- [LLC⁺24] Kaizhan Lin, Jianming Lin, Shiping Cai, Weize Wang, and Chang-An Zhao. Compressed M-SIDH: an instance of compressed SIDH-like schemes with isogenies of highly composite degrees. *DCC*, 92(6):1823–1843, 2024.

- [LWZZ25] Jianming Lin, Weize Wang, Chang-An Zhao, and Yuhao Zheng. Efficient Implementations of Square-root Vélu’s Formulas. *Information Processing Letters*, 190:106580, 2025.
- [MZ24] Hart Montgomery and Mark Zhandry. Full quantum equivalence of group action DLog and CDH, and more. *Journal of Cryptology*, 37(4):39, October 2024.
- [Pei20] Chris Peikert. He Gives C-Sieves on the CSIDH. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part II*, volume 12106 of *LNCS*, pages 463–492, Zagreb, Croatia, May 10–14, 2020. Springer, Cham, Switzerland.
- [Vé71] Jacques Vélu. Isogénies entre courbes elliptiques. *C. R. Acad. Sci., Paris, Sér. A*, 273:238–241, 1971.
- [Wes22] Benjamin Wesolowski. Orientations and the supersingular endomorphism ring problem. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part III*, volume 13277 of *LNCS*, pages 345–371, Trondheim, Norway, May 30 – June 3, 2022. Springer, Cham, Switzerland.
- [WLLZ26] Weize Wang, Yi-Fu Lai, Kaizhan Lin, and Yunlei Zhao. Efficient and Parallel Implementation of Isogeny-based Deterministic Group Actions. Cryptology ePrint Archive, Paper 2026/627, 2026.